

SECURITY AUDIT OF OWASP JUICE SHOP
A Structured Web Application Security Assessment
Major Project Report

Submitted in Partial Fulfillment of the Requirements of
the
Cybersecurity Training and Internship Program

Submitted By
Mehramjot Kaur
B.Tech Computer Science Engineering

Submitted To
YHills

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to YHills for providing me with the opportunity to undertake this project on Security Audit of OWASP Juice Shop. This project provided valuable practical exposure to web application security assessment and enabled me to apply cybersecurity concepts within a controlled laboratory environment.

I am deeply thankful to my Training Mentor for their continuous guidance, encouragement, and technical support throughout the training program. Their valuable insights and constructive feedback greatly contributed to the successful completion of this project.

I would also like to thank the developers and contributors of the OWASP Foundation for providing OWASP Juice Shop, an intentionally vulnerable web application designed for cybersecurity education and hands-on security testing. Additionally, I acknowledge the developers of Burp Suite Community Edition, OWASP ZAP, Nmap, Docker, and Kali Linux, whose tools played an essential role in conducting the security assessment.

Executive Summary

This report presents the results of a structured security assessment conducted on **OWASP Juice Shop**, an intentionally vulnerable web application developed by the Open Worldwide Application Security Project (OWASP) for security training and awareness. The assessment was performed in a controlled laboratory environment to evaluate the application's security posture using standard web application security testing techniques.

The assessment focused on key areas defined within the project scope, including **reconnaissance and application mapping, authentication and session testing, input validation testing, identification of OWASP Top 10 vulnerabilities, and remediation recommendations**. Industry-standard tools such as **Kali Linux, Burp Suite Community Edition, OWASP ZAP, Nmap, Docker, and Mozilla Firefox** were utilized throughout the assessment.

The testing process began with network reconnaissance and application discovery to understand the application's structure, exposed services, and available functionality. Subsequent activities included analyzing authentication mechanisms, examining session management, inspecting HTTP requests and responses, evaluating input handling, and assessing the application against relevant OWASP Top 10 security risks. All observations were documented with supporting evidence, including screenshots, captured HTTP traffic, and scan results.

Based on the assessment findings, a vulnerability matrix was prepared to summarize identified security issues, their severity, associated risks, and recommended remediation measures. The recommendations provided in this report are intended to improve the overall security posture of the application by addressing weaknesses in authentication, session management, input validation, and application configuration.

This assessment was performed **exclusively within an authorized laboratory environment** using OWASP Juice Shop for educational and training purposes. No unauthorized systems or production environments were targeted during the execution of this project. The report demonstrates a systematic approach to web application security assessment while emphasizing ethical testing practices, structured documentation, and the practical application of industry-standard security assessment tools.

Table of Contents

1. Project Overview	7
1.1. Introduction	7
1.2. Assessment Objective	7
1.3. Scope of Assessment	7
1.4. Assessment Tools	8
1.5. Assessment Methodology	8
1.6. Report Organization	9
2. Lab Environment	11
2.1. Overview	11
2.2. Laboratory Environment	11
2.3. Laboratory Architecture	11
2.4. Tools Used	12
2.5. Environment Verification.....	13
2.6. Chapter Summary	16
3. Reconnaissance and Application Mapping.....	17
3.1. Objective	17
3.2. Network Reconnaissance	17
3.3. Application Discovery	20
3.4. HTTP Traffic Analysis.....	22
3.5. REST API Discovery.....	23
3.6. Attack Surface Identification	24
3.7. Chapter Summary.....	24
4. Authentication and Session Testing	26
4.1. Objective	26
4.2. User Registration.....	26
4.3. Login Testing	28
4.4. Session Management Analysis	30
4.5. Authentication Assessment Summary.....	32
5. Input Validation Testing	33
5.1. Objective	33

5.2.	Testing Methodology	33
5.3.	Login Input Validation	33
5.4.	Registration Input Validation	35
5.5.	Search Input Validation	36
5.6.	Address Input Validation	38
5.7.	Profile Update Validation	39
5.8.	Input Validation Summary	40
6.	OWASP Top 10 Security Assessment	41
6.1.	Objective	41
6.2.	Assessment Methodology	41
6.3.	OWASP Top 10 Assessment Summary	41
6.4.	A01 – Broken Access Control.....	42
6.5.	A02 – Cryptographic Failures.....	43
6.6.	A03 – Injection.....	44
6.7.	A05 – Security Misconfiguration	45
6.8.	A07 – Identification and Authentication Failures.....	47
6.9.	Overall Assessment Summary.....	48
6.10.	Chapter Summary	49
7.	Findings and Risk Analysis	50
7.1.	Overview	50
7.2.	Finding F-01.....	50
7.3.	Finding F-02.....	50
7.4.	Finding F-03.....	50
7.5.	Finding F-04.....	50
7.6.	Finding F-05.....	50
7.7.	Risk Summary	50
7.8.	Risk Distribution	50
7.9.	Chapter Summary.....	50
8.	Remediation Recommendations	59
8.1.	Overview	59
8.2.	Remediation Matrix	59

8.3. Detailed Remediation Recommendations	59
8.3.1. Configure Content Security Policy (CSP).....	60
8.3.2. Implement Clickjacking Protection	60
8.3.3. Strengthen Authentication Controls	60
8.3.4. Improve Input Validation	61
8.3.5. Enhance Session Management.....	61
8.4. General Security Best Practices	61
8.5. Remediation Priority.....	62
8.6. Expected Security Improvements	62
8.7. Chapter Summary.....	62
9. Conclusion	63
9.1. Conclusion.....	63
9.2. Key Achievements	63
9.3. Limitations	64
9.4. Future Scope	64
9.5. Final Remarks	64
REFERENCES.....	65
APPENDIX.....	66
Appendix A	66
Appendix B	66
Appendix C	67
Appendix D	67

1. Project Overview

1.1. Introduction

Web applications are among the most frequently targeted assets due to the sensitive information they process and the critical services they provide. Security weaknesses within these applications can lead to unauthorized access, data exposure, session compromise, and other cyber threats. Regular security assessments help identify these weaknesses before they can be exploited and support the implementation of appropriate security controls.

This report presents a structured security assessment of **OWASP Juice Shop**, an intentionally vulnerable web application designed for security training and awareness by the Open Worldwide Application Security Project (OWASP). The assessment was conducted in a controlled laboratory environment using industry-standard security tools and methodologies. The primary focus was to evaluate the application's security posture through reconnaissance, application mapping, authentication and session testing, input validation analysis, and assessment against the OWASP Top 10 Web Application Security Risks.

The assessment was performed exclusively within an authorized environment for educational and training purposes. All findings, observations, and recommendations presented in this report are based on evidence collected during the assessment.

1.2. Assessment Objective

The objective of this security assessment was to evaluate the security posture of the OWASP Juice Shop application through a structured testing methodology. The assessment aimed to identify potential security weaknesses, document observations with supporting evidence, and recommend appropriate remediation measures.

The specific objectives of the assessment were:

- Perform reconnaissance and application mapping.
- Identify exposed services and accessible application components.
- Evaluate authentication and session management mechanisms.
- Assess the application's input validation controls.
- Identify vulnerabilities corresponding to the OWASP Top 10.
- Document findings with supporting evidence.
- Analyze the potential impact of identified vulnerabilities.
- Recommend security improvements based on industry best practices.

1.3. Scope of Assessment

The assessment was limited to the OWASP Juice Shop application deployed locally within a controlled laboratory environment. Testing activities were performed only against the authorized target application and did not involve any external or production systems.

The assessment included the following activities:

- Reconnaissance and application mapping
- Network service identification
- Authentication testing
- Session management testing
- Input validation testing
- HTTP request and response analysis
- Identification of applicable OWASP Top 10 vulnerabilities
- Risk analysis
- Remediation recommendations

The following activities were outside the scope of this assessment:

- Denial-of-Service (DoS) testing
- Testing against external or production systems
- Source code review
- Infrastructure security assessment
- Social engineering attacks

1.4. Assessment Tools

Table 1.1 Tools Used During the Security Assessment

Tool	Version	Purpose
Kali Linux	2026.3	Penetration testing platform
Docker	28.x	Deployment of OWASP Juice Shop
Burp Suite Community Edition	Latest	HTTP interception and traffic analysis
OWASP ZAP	Latest	Automated vulnerability scanning
Nmap	7.99	Network reconnaissance and service detection
Mozilla Firefox	Latest	Web application testing

1.5. Assessment Methodology

The security assessment followed a structured methodology to ensure systematic evaluation of the target application. The assessment began with environment preparation and reconnaissance to understand the application's architecture and exposed services. This was followed by application mapping, authentication testing, session management analysis, input validation testing, and assessment against the OWASP Top 10 security risks. The identified observations were then analyzed to determine their potential impact and appropriate remediation measures.

The overall assessment workflow is illustrated below.

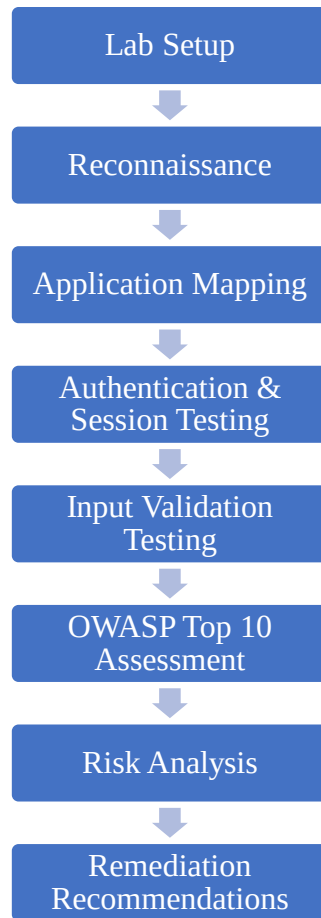


Figure 1.1 Security Assessment Methodology

Figure Description

Figure 1.1 illustrates the structured methodology followed throughout the security assessment. Each phase builds upon the previous one, ensuring that information gathered during reconnaissance supports subsequent testing activities and contributes to a comprehensive evaluation of the application's security posture.

1.6. Report Organization

This report is organized into the following sections:

Table 1.2 Report Organization

Section	Description
Executive Summary	Overview of the assessment and key objectives
Project Overview	Introduction, objectives, scope, tools, and methodology
Lab Environment	Laboratory setup and tool configuration

Section	Description
Reconnaissance & Application Mapping	Network discovery, application exploration, and attack surface identification
Authentication & Session Testing	Analysis of authentication and session management
Input Validation Testing	Evaluation of application input handling
OWASP Top 10 Assessment	Assessment against applicable OWASP security categories
Findings & Risk Analysis	Identified observations, evidence, severity, and impact
Vulnerability Matrix	Summary of identified security findings
Remediation Recommendations	Recommended security improvements
Conclusion	Summary of the overall assessment

2. Lab Environment

2.1. Overview

A dedicated laboratory environment was established to conduct the security assessment in a safe and controlled manner. The environment was designed to isolate all testing activities from production systems while providing a realistic platform for evaluating web application security. The assessment was performed using Kali Linux running inside Oracle VirtualBox, with OWASP Juice Shop deployed locally as a Docker container. All HTTP traffic between the web browser and the application was intercepted using Burp Suite Community Edition to facilitate detailed request and response analysis.

The laboratory setup ensured that all testing activities remained within an authorized environment, supporting ethical security testing practices.

2.2. Laboratory Environment

The assessment environment consisted of a Windows host operating system running Oracle VirtualBox. Kali Linux was configured as the guest operating system and served as the primary penetration testing platform. OWASP Juice Shop was deployed locally using Docker and accessed through Mozilla Firefox. Burp Suite Community Edition was configured as an intercepting proxy to capture and analyze HTTP requests and responses generated during the assessment.

Table 2.1 Laboratory Environment Configuration

Component	Description
Host Operating System	Microsoft Windows
Virtualization Platform	Oracle VirtualBox
Guest Operating System	Kali Linux 2026.3
Target Application	OWASP Juice Shop
Deployment Method	Docker Container
Browser	Mozilla Firefox
Proxy Tool	Burp Suite Community Edition
Network Scanner	Nmap
Vulnerability Scanner	OWASP ZAP

2.3. Laboratory Architecture

The laboratory architecture used during the assessment is illustrated in Figure 2.1.

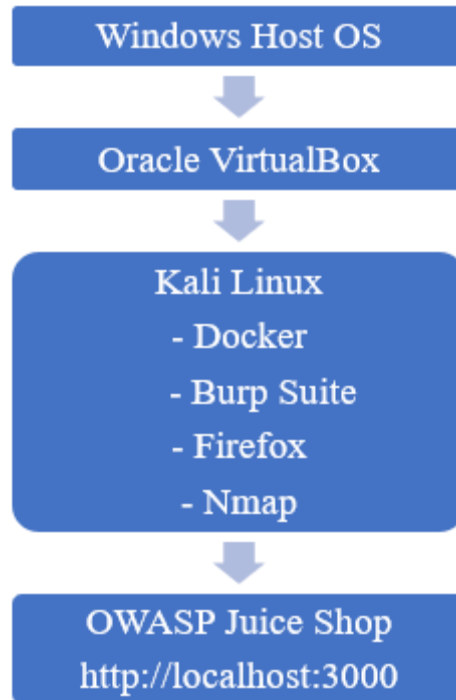


Figure 2.1 Laboratory Architecture of the Security Assessment Environment

Figure Description

Figure 2.1 illustrates the architecture of the testing environment used during the security assessment. The Windows host system runs Oracle VirtualBox, which hosts the Kali Linux virtual machine. Kali Linux serves as the primary testing platform and includes Docker for deploying OWASP Juice Shop, Burp Suite Community Edition for HTTP interception, Mozilla Firefox for interacting with the application, and Nmap for network reconnaissance. The OWASP Juice Shop application is hosted locally and accessed through <http://localhost:3000>, providing an isolated environment for ethical security testing.

2.4. Tools Used

The security assessment was performed using industry-standard open-source and community tools. Each tool was selected based on its specific role within the assessment methodology.

Table 2.2 Security Assessment Tools

Tool	Purpose
Kali Linux	Penetration testing operating system
Docker	Deployment of OWASP Juice Shop
Burp Suite Community Edition	Interception and analysis of HTTP/HTTPS traffic

Tool	Purpose
Mozilla Firefox	Web browser used for interacting with the application
Nmap	Network reconnaissance and service detection
OWASP ZAP	Automated web application vulnerability scanning

2.5. Environment Verification

Before initiating the security assessment, the testing environment was verified to ensure that all required tools and services were functioning correctly. Verification included confirming the operating system version, Docker installation, successful deployment of the OWASP Juice Shop container, availability of the target application, and correct configuration of Burp Suite and Nmap.

Table 2.3 Environment Verification

Verification Item	Status
Kali Linux Installed	Verified
Docker Installed	Verified
OWASP Juice Shop Running	Verified
Burp Suite Operational	Verified
Nmap Installed	Verified
Firefox Proxy Configuration	Verified

```
(kali@kali)~$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.3"
VERSION="2026.3"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
```

Figure 2.2 Kali Linux Environment

Figure Description

Figure 2.2 shows the Kali Linux operating system used as the primary platform for conducting the security assessment.

```
(kali@kali)-[~]
$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed
```

Figure 2.3 Docker Installation Verification

Figure Description

Figure 2.3 verifies that Docker was successfully installed and available for deploying the OWASP Juice Shop application.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c42013462c53	bkimminich/juice-shop	"/nodejs/bin/node /j..."	8 minutes ago	Up 8 minutes	0.0.0.0:3000→3000/tcp, [::]:3000→3000/tcp	juice-shop

Figure 2.4 OWASP Juice Shop Docker Container

Figure Description

Figure 2.4 confirms that the OWASP Juice Shop Docker container was successfully deployed and running during the assessment.

```
(kali@kali)-[~]
$ nmap --version
Nmap version 7.99 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 openssl-3.6.3 libssh2-1.11.1 libz-1.3.1 libpcr2-10.46 libpcap-1.10.6 nmap-libdnet-1.18.0 ipv6
Compiled without:
Available nsock engines: epoll poll select
```

Figure 2.5 Nmap Installation Verification

Figure Description

Figure 2.5 confirms the availability of Nmap, which was later used for network reconnaissance and service detection.

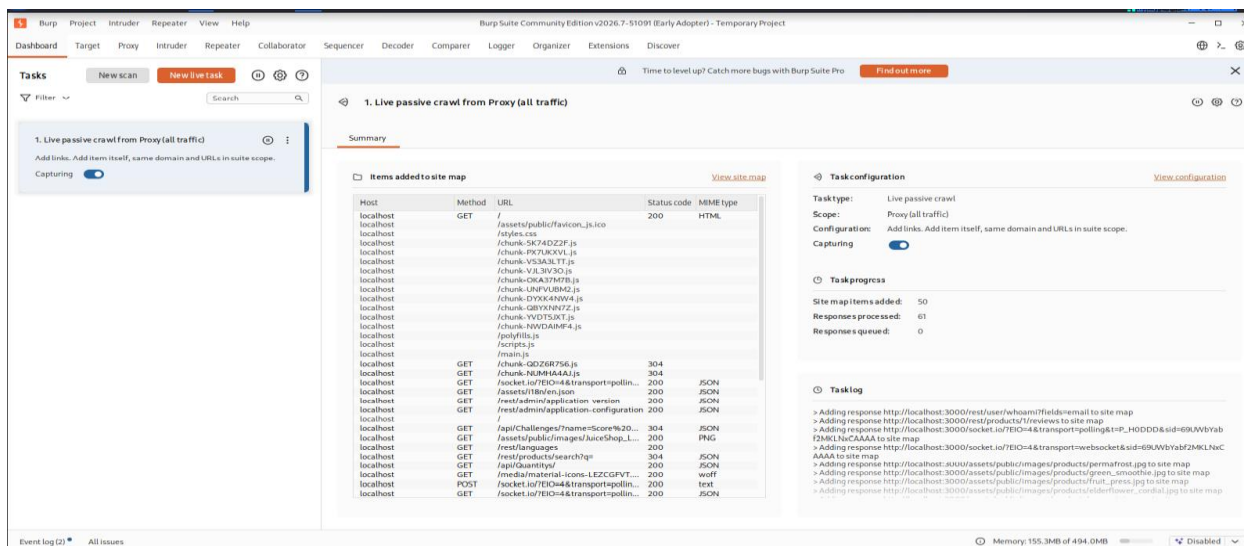


Figure 2.6 Burp Suite Community Edition

Figure Description

Figure 2.6 presents the Burp Suite Community Edition interface used to intercept and analyze HTTP requests and responses during the assessment.

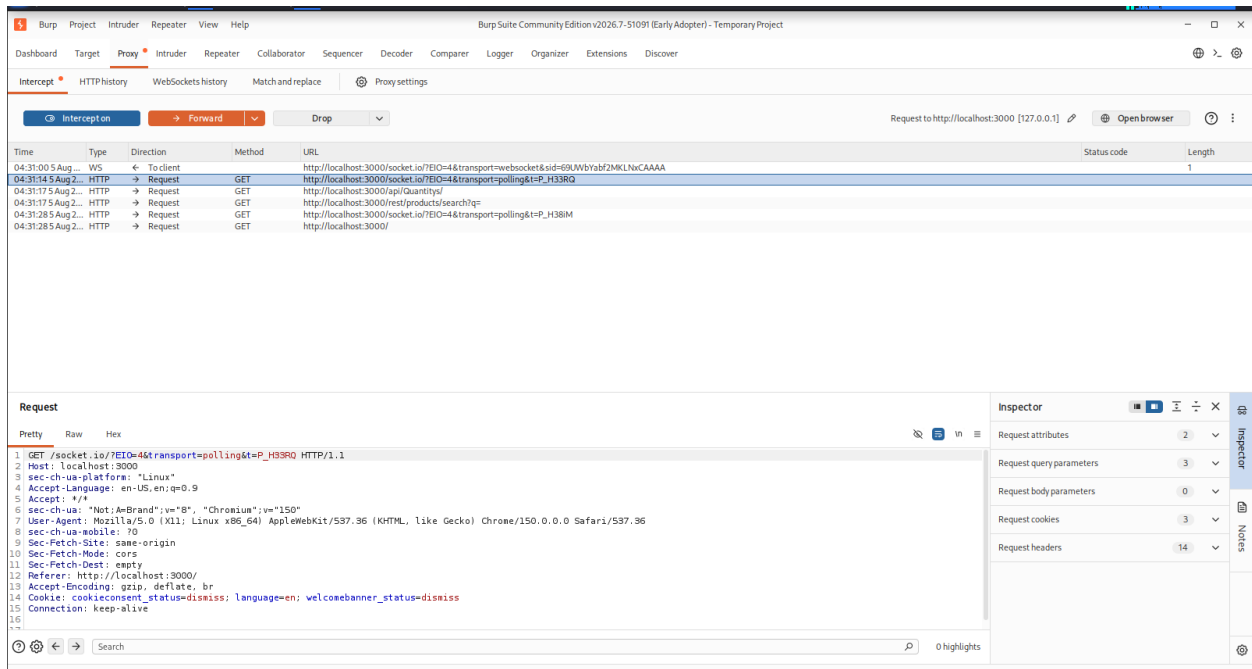


Figure 2.7 Burp Suite Intercept

Figure Description

Figure 2.7 demonstrates successful interception of an HTTP request using Burp Suite, confirming that browser traffic was correctly routed through the proxy.

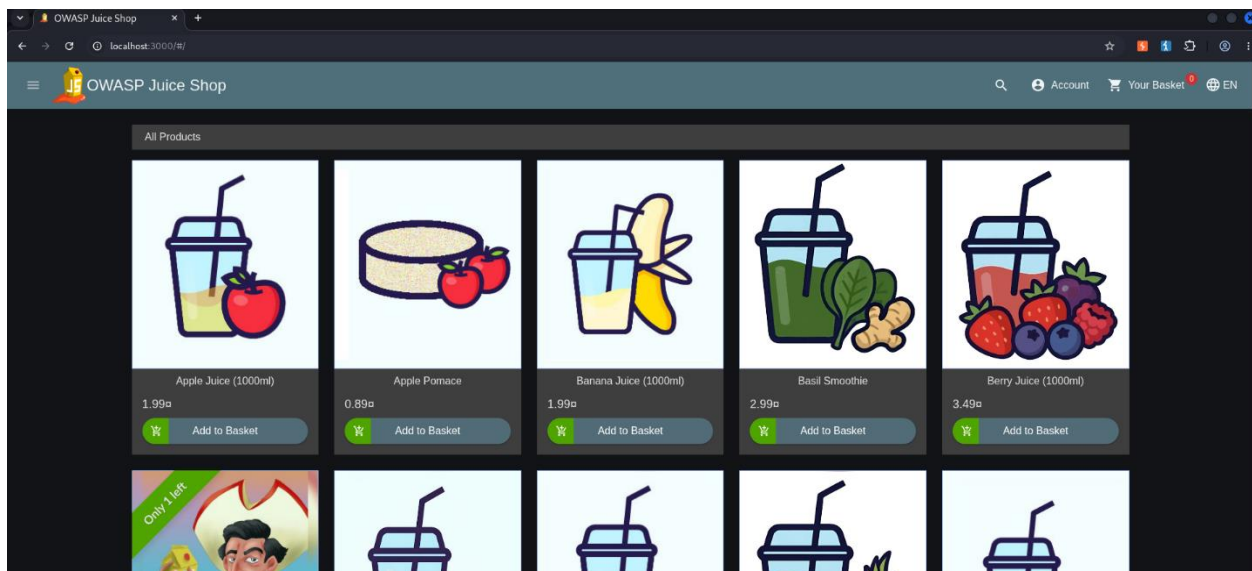


Figure 2.8 OWASP Juice Shop Home Page

Figure Description

Figure 2.8 shows the OWASP Juice Shop application running locally and accessible through `http://localhost:3000`, confirming successful deployment and readiness for assessment.

2.6. Chapter Summary

A dedicated laboratory environment was successfully configured to conduct the security assessment of OWASP Juice Shop. The assessment platform consisted of Kali Linux running inside Oracle VirtualBox, with the target application deployed locally using Docker. Burp Suite Community Edition was configured as an intercepting proxy, and Nmap was verified for reconnaissance activities. Environment verification confirmed that all required tools and services were operational before initiating security testing. This controlled setup provided the foundation for the reconnaissance and application mapping activities presented in the next chapter.

3. Reconnaissance and Application Mapping

3.1. Objective

The objective of this phase was to gather information about the target application before performing detailed security testing. Reconnaissance was conducted to identify exposed network services, verify application availability, understand the application's functionality, discover accessible pages and REST API endpoints, and identify potential input points for subsequent security testing.

Both **Nmap** and **Burp Suite Community Edition** were used during this phase. Nmap was used for network service discovery, while Burp Suite was used to inspect HTTP traffic and understand the application's communication with the backend.

3.2. Network Reconnaissance

Network reconnaissance was performed using **Nmap (Network Mapper)** to verify that the target application was accessible and to identify any exposed network services. Two different scans were conducted during this phase:

- **Service Version Detection Scan (nmap -sV localhost)**
- **Aggressive Scan (nmap -A localhost)**

The service version detection scan was used to identify the service running on the target port, while the aggressive scan collected additional information such as HTTP response headers, service fingerprints, and operating system detection.

The following commands were executed:

```
nmap -sV localhost
```

```
nmap -A localhost
```

The scans confirmed that the application was accessible through **TCP port 3000**. Although Nmap classified the detected service as "**ppp?**", the HTTP response clearly identified the target as the **OWASP Juice Shop** application. The aggressive scan successfully retrieved HTTP response headers and HTML content, confirming that the application was operational and accessible for further testing.

Table 3.1 Nmap Service Detection Results

Parameter	Observation
Target Host	localhost (127.0.0.1)
Scan Type	Service Version Detection (-sV)
Open Port	3000/TCP
Port State	Open

Parameter	Observation
Security Headers Observed	X-Frame-Options, X-Content-Type-Options, Access-Control-Allow-Origin
Operating System Detection	No exact OS match
Network Distance	0 hops

[illegible]

Figure 3.2 Nmap Aggressive Scan Results

Figure Description

Figure 3.2 presents the results of the aggressive Nmap scan. The scan retrieved HTTP response headers, HTML content, and application metadata, confirming that the target was the OWASP Juice Shop application. Operating system detection was attempted but did not produce an exact match because the application was deployed inside a Docker container.

3.3. Application Discovery

Following network reconnaissance, the application was explored manually to understand its functionality and identify the primary features available to users. This activity helped define the overall attack surface and identify components requiring further security evaluation.

The following application components were identified during manual exploration.

Table 3.3 Identified Application Components

Component	Description	Authentication Required
Home	Displays available products	No
Login	User authentication	No
Register	New user registration	No
Search	Product search functionality	No
Basket	Shopping cart management	Yes
User Profile	User account management	Yes
Orders	View previous orders	Yes
Contact/Feedback	Customer communication	No

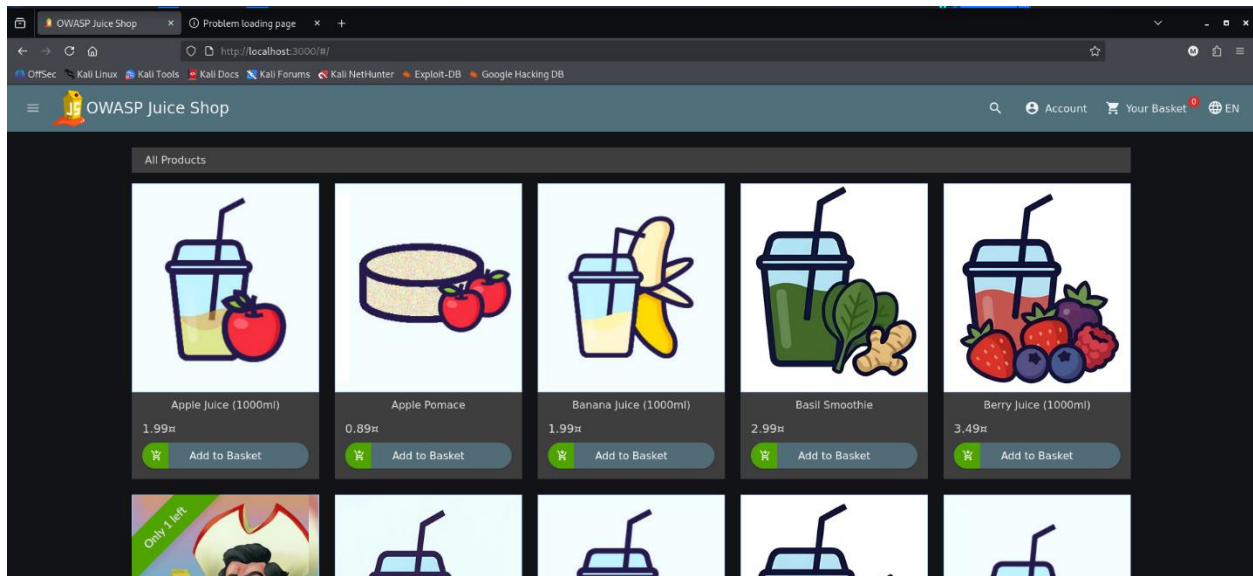


Figure 3.3 OWASP Juice Shop Home Page

Figure Description

Figure 3.3 shows the OWASP Juice Shop homepage, which serves as the primary interface for browsing products and accessing application functionality.

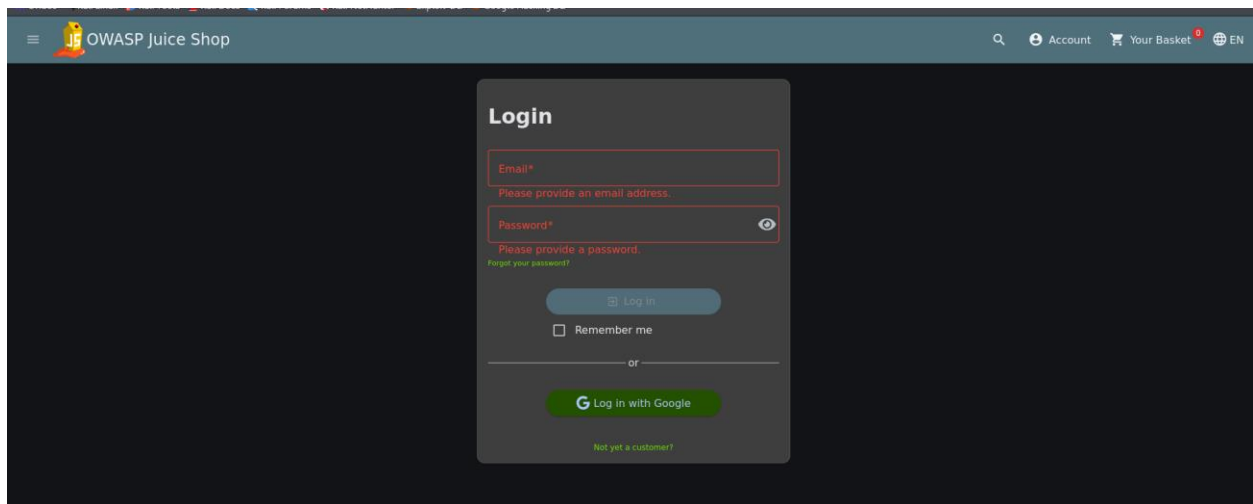
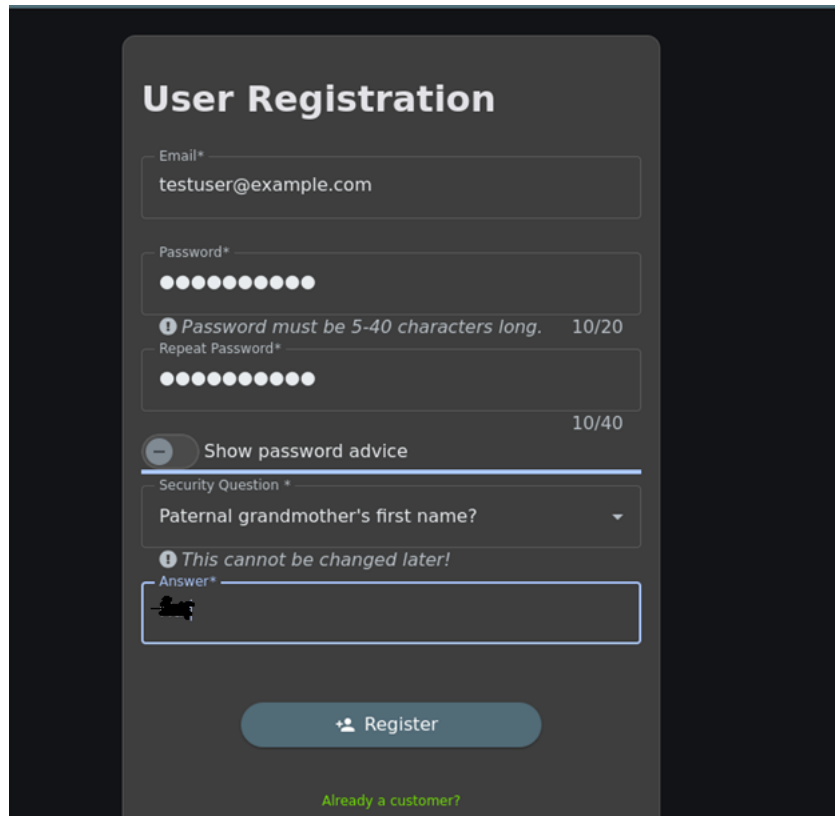


Figure 3.4 Login Page

Figure Description

Figure 3.4 presents the user authentication interface used for logging into the application.

A user registration form titled "User Registration" on a dark background. The form contains several input fields: "Email*" with the value "testuser@example.com", "Password*" with masked characters and a strength indicator "10/20" and a note "Password must be 5-40 characters long.", "Repeat Password*" with masked characters and a strength indicator "10/40", a "Show password advice" toggle, "Security Question *" with a dropdown menu showing "Paternal grandmother's first name?", and "Answer*" with a masked input field and a note "This cannot be changed later!". At the bottom is a "Register" button with a user icon and a link "Already a customer?".

User Registration

Email*
testuser@example.com

Password*
●●●●●●●●
1 Password must be 5-40 characters long. 10/20

Repeat Password*
●●●●●●●●
10/40

Show password advice

Security Question *
Paternal grandmother's first name?

This cannot be changed later!

Answer*
●●●●●●●●

Register

Already a customer?

Figure 3.5 Registration Page

Figure Description

Figure 3.5 illustrates the registration interface through which new users can create an account for accessing authenticated features.

3.4. HTTP Traffic Analysis

Burp Suite Community Edition was configured as an intercepting proxy to capture HTTP requests exchanged between Mozilla Firefox and the OWASP Juice Shop application.

Analysis of HTTP traffic provided insight into request methods, response codes, application endpoints, and communication between the client and the server. The captured traffic also assisted in identifying REST API endpoints that would be analyzed in later phases of the assessment.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Time	Listener port	Start response
1	http://localhost:3000	GET	/			200	10370	HTML		OWASP Juice Shop			127.0.0.1	04:17:01.5...	8080	172
3	http://localhost:3000	GET	/			200	326	text	io/				127.0.0.1	04:18:19.5...	8080	172
20	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t...			200	35940	JSON	json				127.0.0.1	04:18:29.5...	8080	20
21	http://localhost:3000	GET	/assets/18h/en.json			200	404	JSON					127.0.0.1	04:18:30.5...	8080	87
22	http://localhost:3000	GET	/rest/admin/application-version			200	23966	JSON					127.0.0.1	04:18:30.5...	8080	479
23	http://localhost:3000	GET	/rest/admin/application-configuration			200	862	JSON					127.0.0.1	04:18:30.5...	8080	758
24	http://localhost:3000	GET	/api/challenges/?name=Score%20Board			200	4603	JSON					127.0.0.1	04:18:31.5...	8080	3487
25	http://localhost:3000	GET	/rest/languages			200	304						127.0.0.1	04:18:31.5...	8080	52
26	http://localhost:3000	GET	/rest/admin/application-version			304	306						127.0.0.1	04:18:31.5...	8080	356
28	http://localhost:3000	GET	/api/challenges/?name=Score%20Board			304	305						127.0.0.1	04:18:31.5...	8080	4435
29	http://localhost:3000	GET	/rest/admin/application-configuration			304	306						127.0.0.1	04:18:31.5...	8080	633
30	http://localhost:3000	GET	/rest/admin/application-configuration			304	306						127.0.0.1	04:18:33.5...	8080	1458
31	http://localhost:3000	GET	/api/quantity			200	8016	JSON					127.0.0.1	04:18:33.5...	8080	2653
32	http://localhost:3000	GET	/rest/products/search?q=			200	16952	JSON					127.0.0.1	04:18:34.5...	8080	1770
35	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&t...			200	215	HTML	io/				127.0.0.1	04:18:37.5...	8080	133
36	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t...			200	283	text	io/				127.0.0.1	04:18:37.5...	8080	133
37	http://localhost:3000	GET	/socket.io/?EIO=4&transport=websock...			101	129						127.0.0.1	04:18:39.5...	8080	23
53	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t...			200	230	text	io/				127.0.0.1	04:18:40.5...	8080	746
55	http://localhost:3000	GET	/rest/user/whom?fields=email			200	384	JSON					127.0.0.1	04:18:51.5...	8080	653
56	http://localhost:3000	GET	/rest/products/1/reviews			200	781	JSON					127.0.0.1	04:18:51.5...	8080	638
57	http://localhost:3000	GET	/rest/products/1/reviews			304	305						127.0.0.1	04:18:52.5...	8080	40
58	http://localhost:3000	GET	/rest/products/1/reviews			304	305						127.0.0.1	04:18:53.5...	8080	68
59	http://localhost:3000	GET	/rest/admin/application-configuration			304	306						127.0.0.1	04:19:03.5...	8080	50
60	http://localhost:3000	GET	/api/quantity			304	306						127.0.0.1	04:19:06.5...	8080	371
61	http://localhost:3000	GET	/rest/products/search?q=			304	306						127.0.0.1	04:19:06.5...	8080	311
62	http://localhost:3000	GET	/api/quantity			304	306						127.0.0.1	04:19:16.5...	8080	196
63	http://localhost:3000	GET	/rest/products/search?q=			304	306						127.0.0.1	04:19:16.5...	8080	142

Figure 3.6 Burp Suite HTTP History

Figure Description

Figure 3.6 displays the HTTP History recorded by Burp Suite while interacting with the application. The captured requests provide visibility into the application's communication flow and support further security analysis.

3.5. REST API Discovery

Analysis of the captured HTTP traffic identified several REST API endpoints used by the application to retrieve data and process user requests.

Table 3.4 Identified Application Endpoints from Burp Suite HTTP History

HTTP Method	Endpoint	Description
GET	/	Loads the OWASP Juice Shop homepage
GET	/index.html	Retrieves the main application page
GET	/manifest.webmanifest	Loads Progressive Web App (PWA) manifest
GET	/assets/public/images/products/...	Retrieves product images displayed on the homepage

HTTP Method	Endpoint	Description
GET	/rest/admin/application-configuration	Retrieves application configuration information
GET	/rest/admin/application-version	Retrieves the application version
GET	/rest/languages	Retrieves supported application languages
GET	/rest/products/search	Retrieves or searches available products
GET	/rest/products/reviews	Retrieves product reviews
GET	/rest/saveLoginIp	Stores or updates login IP information
GET	/rest/deluxe-membership	Retrieves deluxe membership information

Note: The table above is based on the HTTP requests captured during the reconnaissance phase in Burp Suite. Additional endpoints may appear during authentication, shopping, profile management, and other user interactions in later assessment phases.

3.6. Attack Surface Identification

The application contains several user-accessible input points that accept data from users. These interfaces represent potential attack surfaces and were identified for further evaluation during the input validation phase.

Table 3.5 Identified Input Points

Input Location	Purpose
Login Form	User authentication
Registration Form	New account creation
Search Bar	Product search
Contact Form	Customer feedback
Address Form	Shipping information
User Profile	Profile management

3.7. Chapter Summary

This chapter documented the reconnaissance and application mapping activities performed during the security assessment of OWASP Juice Shop. Network reconnaissance using Nmap confirmed that the application was accessible through **TCP port 3000**, while service detection and aggressive scanning provided detailed information about the exposed web service and HTTP response

characteristics. Manual exploration and HTTP traffic analysis using Burp Suite enabled the identification of key application components, REST API endpoints, and primary user input locations.

The information gathered during this phase established a clear understanding of the application's attack surface and provided the foundation for the authentication testing, session management analysis, and input validation assessment presented in the subsequent chapters.

4. Authentication and Session Testing

4.1. Objective

The objective of this phase was to evaluate the authentication mechanism and session management implemented by the OWASP Juice Shop application. The assessment focused on user registration, login functionality, session creation, authentication token handling, session persistence, and logout behavior. Burp Suite Community Edition was used to intercept and analyze HTTP requests and responses exchanged between the client and the application during the authentication process.

4.2. User Registration

A new user account was created using the application's registration functionality. Burp Suite was configured as an intercepting proxy to capture the registration request before it was forwarded to the server. The intercepted request contained the registration details submitted by the user, including the email address, password, selected security question, and security answer.

The registration process completed successfully, confirming that the application accepted valid user input and created a new account.

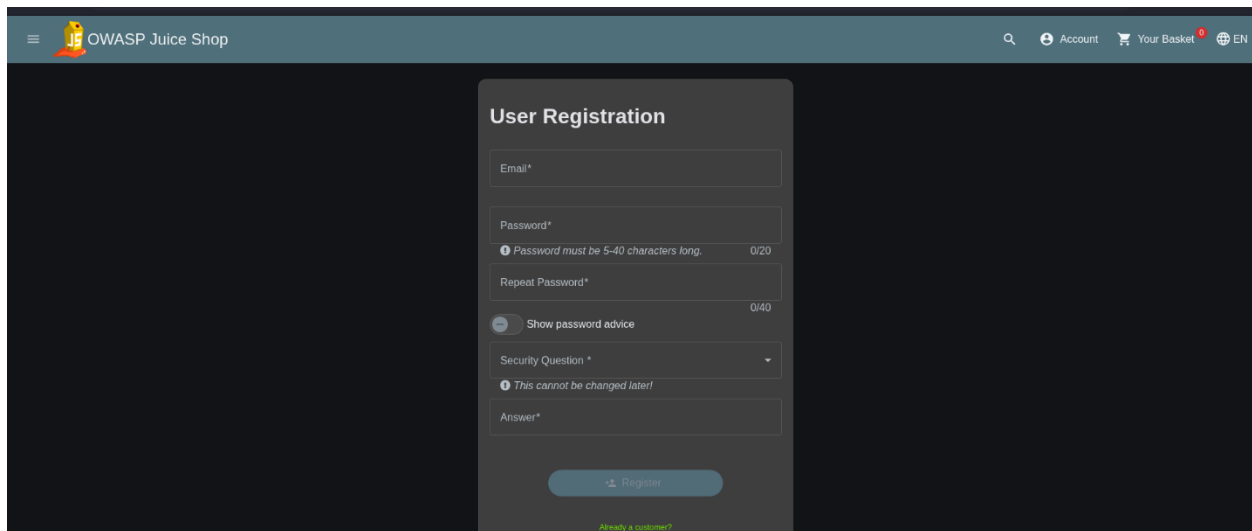
The image is a screenshot of the OWASP Juice Shop web application's user registration interface. The page has a dark theme. At the top, there is a navigation bar with the OWASP Juice Shop logo on the left and links for 'Account', 'Your Basket', and 'EN' on the right. The main content area is a dark gray box with the title 'User Registration' at the top. Below the title, there are several input fields: 'Email*', 'Password*', 'Repeat Password*', 'Security Question *', and 'Answer*'. The 'Password*' field has a character count '0/20' and a hint 'Password must be 5-40 characters long.'. The 'Repeat Password*' field has a character count '0/40'. There is a 'Show password advice' toggle switch. Below the 'Security Question *' dropdown, there is a hint 'This cannot be changed later!'. At the bottom of the form is a 'Register' button. Below the button, there is a link 'Already a customer?'. The background of the page is black.

Figure 4.1 User Registration Page

Figure Description

Figure 4.1 shows the registration interface of OWASP Juice Shop, where a new user account was created for authentication testing.

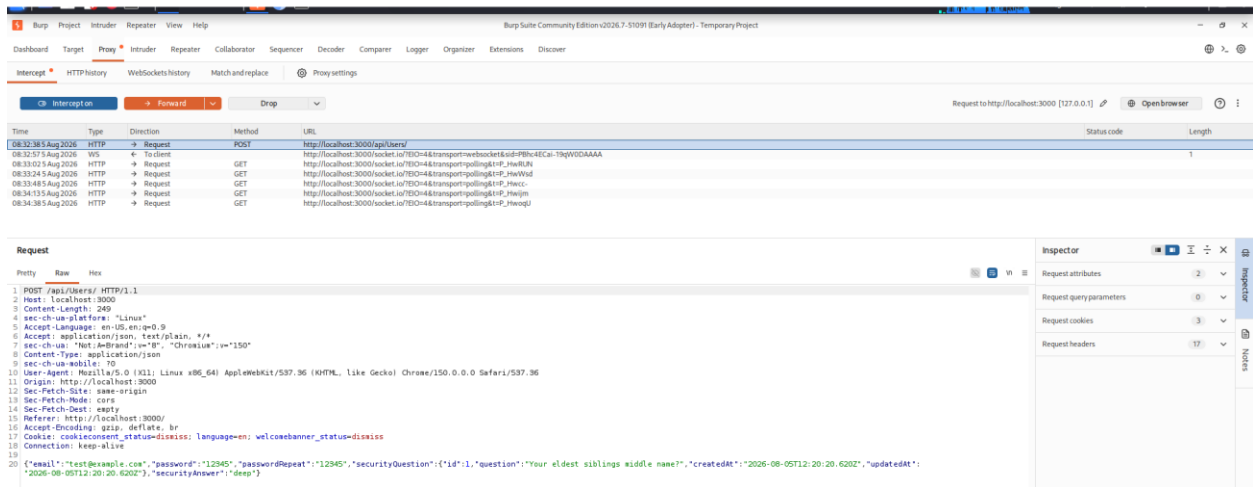


Figure 4.2 User Registration Request Captured in Burp Suite

Figure Description

Figure 4.2 illustrates the intercepted HTTP **POST** request sent to the `/api/Users/` endpoint during user registration. The request contains the registration details, including the email address, password, security question, and security answer submitted by the user.

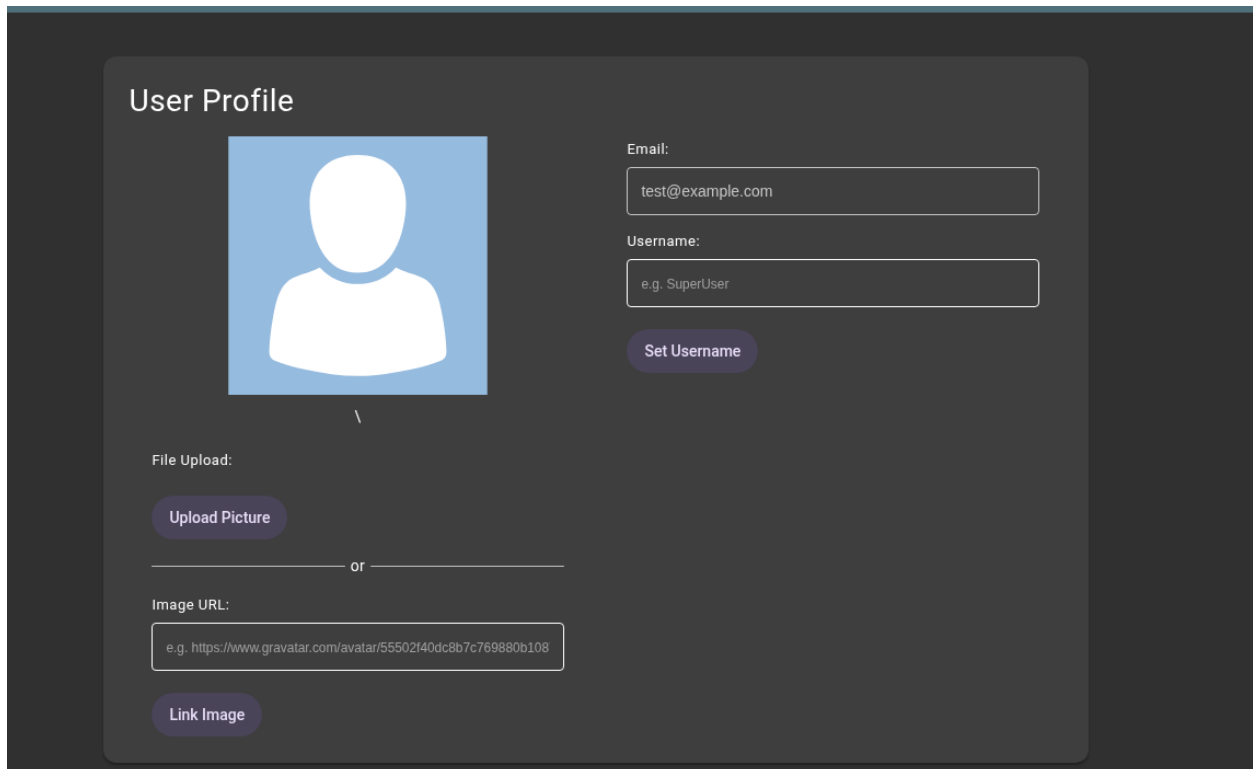


Figure 4.3 Successful User Registration

Figure Description

Figure 4.3 confirms the successful creation of a new user account following the submission of valid registration information.

4.3. Login Testing

After successful registration, the newly created account was used to authenticate with the application. Burp Suite intercepted the login request, allowing inspection of the HTTP request before it was forwarded to the server.

The login request was submitted to the /rest/user/login endpoint using the HTTP **POST** method. Upon successful authentication, the application granted access to authenticated functionality.

Table 4.1 Authentication Request Details

Parameter	Observation
HTTP Method	POST
Login Endpoint	/rest/user/login
Authentication Method	Email and Password
Request Format	JSON
Result	Authentication Successful

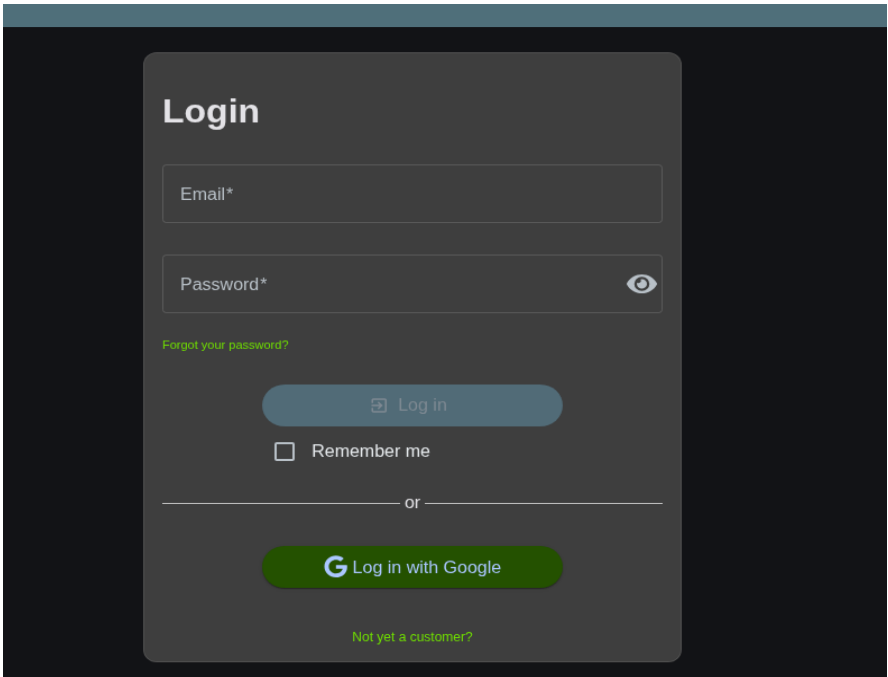


Figure 4.4 User Login Page

Figure 4.4 presents the login interface used to authenticate the registered user.



Figure 4.5 shows the intercepted HTTP **POST** request submitted to the `/rest/user/login` endpoint. The request includes the user's email address and password in JSON format.



Figure Description

Figure 4.6 displays the successful authentication response received from the server after valid credentials were submitted.

4.4. Session Management Analysis

Following successful authentication, the application's session management mechanism was analyzed. The assessment focused on authentication token handling, request authorization, session persistence, and logout behavior.

Subsequent authenticated requests were inspected using Burp Suite to determine how the application maintained the authenticated session. The analysis confirmed that the application used token-based authentication for accessing protected resources.

Table 4.2 Session Management Observations

Assessment Item	Observation
Authentication Mechanism	Token-based authentication
Authenticated Requests	Successfully observed
Session Persistence	Maintained after successful login
Authorization Header	Present in authenticated requests
Logout API Endpoint	Not observed
Logout Behaviour	Client-side logout performed successfully

Note: During testing, no dedicated logout endpoint was observed in the captured HTTP traffic. The application handled logout by clearing the client-side authentication state rather than issuing a server-side logout request. This behavior is consistent with JWT-based authentication mechanisms used by modern single-page applications.

```
{
  "authentication": {
    "token":
      "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpb7ImkIj0yNCwidXNlcm5hbWUiOiIiLCJlbWFPbCI6InRlc3RAZXhhbXBsZS5jb20iLCJwYXNkd29yZCI6IjgyN2NjYjBLZWE4YTcwNmM0YzMOYTE2ODkxZjg0ZTdiIiwicm9sZSI6ImNlc3RvbWVvIiwiaGVhZGVkIjpbVG9rZW4iOiIiLCJ0eXN0TG9naW5JcCI6IiIsInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGllL2ltYXdIcy9lcGxvYWRzL2RlZmF1bHouc3ZnIiwidG90cFNLy3JldCI6IiIsImIzQWNOaXZLIjpb0cnVLCJjcmVhdGVkQXQiOiIyMDI2LTA4LTA1IDEyOjM2OjI3LjYzNyArMDA6MDAiLCJlcGRhdGVkQXQiOiIyMDI2LTA4LTA1IDEyOjM4OjU4LjEzMyArMDA6MDAiLCJkZWxldGVkQXQiOiIyMDI2LTA4LTA1IDEyOjYsImIhdCI6MTc4NTkzMzc1MH0.E5ua5HrCHEmgiJ8nfDOR800ZuWB2eAzMZR1_JgV8-545GXjRMWrfPhOTLTlnHwu5UJo1IYznqTYVnp8WUeJxx-7e9kdmqYSb6LN0mw7iiAqZiCin--GW6L3v6d0b-fI-1UfzHUZaRf1NZqzle4YUgElwa141FrIgEgnGCKaStFU",
    "bid": 6,
    "umail": "test@example.com"
  }
}
```

Figure 4.7 Authentication Token / Session Information

Figure Description

Figure 4.7 presents the authentication token or session-related information observed after successful user authentication. The application used token-based authentication to maintain the authenticated session.

```
sec-ch-ua-platform: "Linux"
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpb7ImkIjoiYm9wZXNlcm5hbWUiOiIiLCJlbWVpbCI6InRlc3RAZXhhbXBsZS5jb20iLCJwYXNzd29yZCI6IjgyN2NjYjB1ZWE4YTcwNmM0YzMOYTE2ODkxZjg0ZTdiIiwicm9sZSI6ImNlc3RvbWVvIiwidGVzdXhlVG9rZW4iOiIiLCJsYXN0TG9naW5jcCI6IjAuMC4wLjAiLCJwcm9maWx1SW1hZ2UiOiIvYXNzZXRzL3B1Ym9pYy9pbWFnZXNvdXBsZ2Fkcy9kZWZhdWx0LnN2ZyIsInRvdHBTZWNyZXQiOiIiLCJpc0FjdG12ZSI6dHJ1ZSwiY3JlYXRlZEF0IjoiMjAyNi0wOC0wNSAxMj0zNj0yNy42MzcgKzAwOjAwIiwidXBkYXRlZEF0IjoiMjAyNi0wOC0wNSAxMj0zNj0yNy42MzcgKzAwOjAwIiwidGVsZXRLZEF0IjpbudWxsfsWlklIjo2LCJpYXQiOjE3ODU5MzMOjNTN9.DXavsEgQtOHCEsIE4qxVJ9hsbKwxbACN-54JWPzYf5U6GSR54UJfELGaZzwdFWovmp8Wn604zCaGzvJTsHoI-xarQ3aTRUTvI419Um1ld_1qtpsLdivYrJWSL6encIV_BZLncvWE_3bGzuda8iL7zSMvd4KSH4g50RidBX9Nn6U
```

Figure 4.8 Authorization Header in Authenticated Request

Figure Description

Figure 4.8 illustrates an authenticated HTTP request containing the authorization information used to access protected application resources.

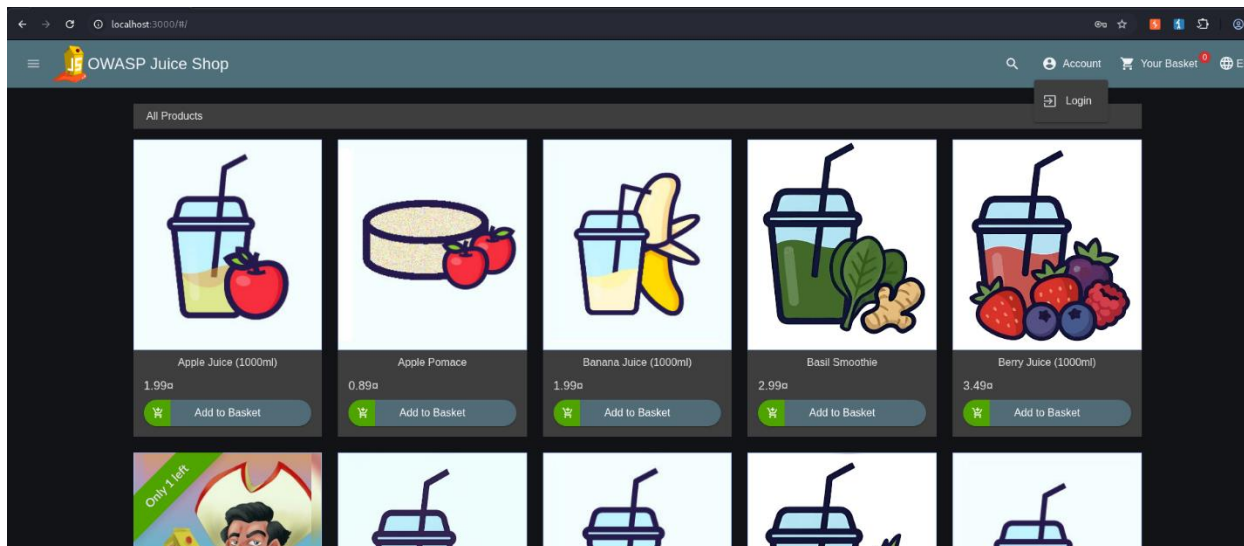


Figure 4.9 User Logout

Figure Description

Figure 4.9 shows the application after the user initiated the logout operation. No dedicated logout API endpoint was observed during HTTP traffic analysis. The application performed logout by clearing the client-side authentication state and returning the user to an unauthenticated state.

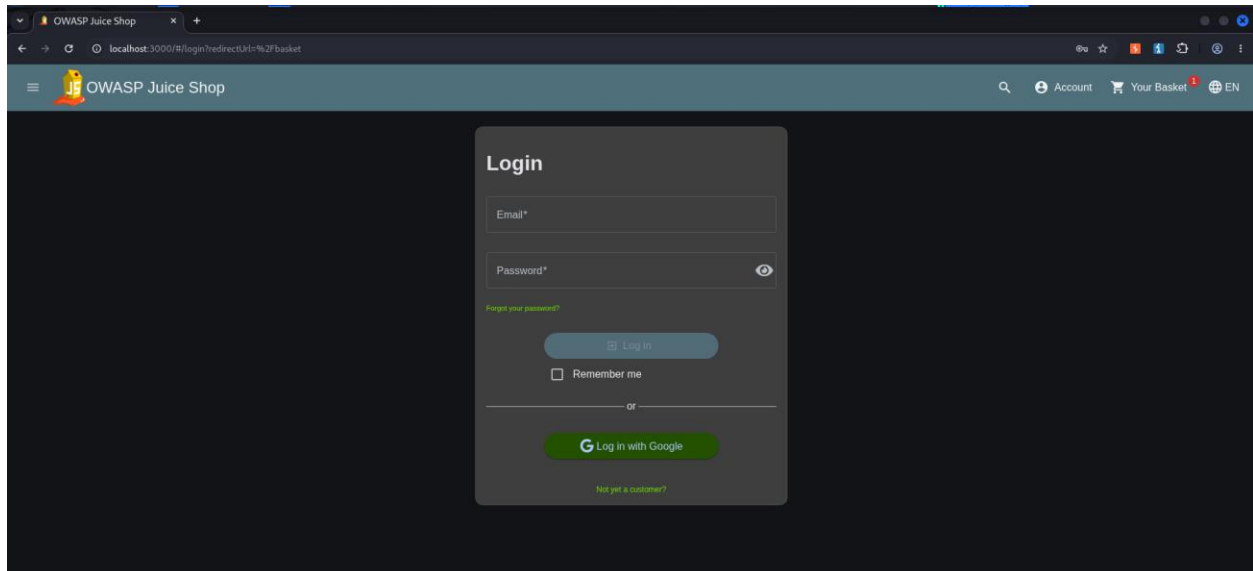


Figure 4.10 Session Verification After Logout

Figure Description

Figure 4.10 demonstrates that authenticated functionality was no longer accessible after logout, confirming that the previous authentication state had been successfully terminated.

4.5. Authentication Assessment Summary

The authentication mechanism of OWASP Juice Shop was successfully evaluated through user registration, login, session analysis, and logout verification. Burp Suite enabled detailed inspection of the HTTP requests exchanged during authentication, confirming that credentials were transmitted using JSON-formatted POST requests. The application successfully authenticated valid users and maintained authenticated sessions using a token-based mechanism.

No dedicated logout endpoint was observed during testing. Instead, the application performed logout by removing the client-side authentication state, which is consistent with the behavior of many modern single-page applications using JWT-based authentication. The authenticated session was effectively terminated, and access to protected functionality required re-authentication.

5. Input Validation Testing

5.1.Objective

The objective of this phase was to evaluate how OWASP Juice Shop processes and validates user-supplied input across different application components. Proper input validation is essential for preventing common web application attacks such as SQL Injection (SQLi), Cross-Site Scripting (XSS), and malformed input processing.

Testing focused on observing the application's behavior when valid, invalid, and specially crafted inputs were submitted through various user-accessible forms. Burp Suite Community Edition was used to intercept and analyze HTTP requests and responses throughout the assessment.

5.2. Testing Methodology

Input validation testing was performed on multiple application components by submitting normal input, malformed input, and common security testing payloads. Each request was intercepted using Burp Suite to inspect the transmitted parameters and evaluate the application's response.

The following components were assessed:

- Login Form
- User Registration
- Product Search
- Address Management
- User Profile

The assessment was conducted within the authorized laboratory environment without attempting to exploit vulnerabilities.

5.3. Login Input Validation

The login functionality was evaluated using invalid authentication credentials to determine whether the application handled authentication failures securely.

Burp Suite successfully intercepted the login request before transmission to the server. After forwarding the request, the application rejected the invalid credentials and displayed a generic authentication error message.

Table 5.1 Login Validation Result

Test Case	Input	Observation	Result
Invalid Login	Incorrect password	Generic authentication error displayed	Passed

Request

Pretty Raw Hex

```
1 POST /rest/user/login HTTP/1.1
2 Host: localhost:3000
3 Content-Length: 49
4 sec-ch-ua-platform: "Linux"
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/plain, */*
7 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
8 Content-Type: application/json
9 sec-ch-ua-mobile: ?0
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
11 Origin: http://localhost:3000
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://localhost:3000/
16 Accept-Encoding: gzip, deflate, br
17 Cookie: cookieconsent_status=dismiss; language=en; welcomebanner_status=dismiss
18 Connection: keep-alive
19
20 {"email":"test@example.com","password":"7863243"}
```

Figure 11 Invalid Login Request Captured in Burp Suite

Figure Description

Figure 5.1 illustrates the intercepted HTTP POST request submitted to the /rest/user/login endpoint containing invalid user credentials.

Login

Invalid email or password.

Email*

test@example.com

Password*

.....

Forgot your password?

Log in

☐ Remember me

or

Log in with Google

Not yet a customer?

Figure 12 Authentication Failure Message

Figure Description

Figure 5.2 shows the application's response after an authentication attempt using invalid credentials. The application displayed the message **"Invalid email or password."** without revealing whether the email address or password was incorrect, thereby reducing the possibility of user enumeration.

5.4. Registration Input Validation

The registration form was evaluated using invalid email formats and weak passwords to determine whether appropriate validation mechanisms were enforced.

Testing confirmed that the application validates the email format before allowing registration. Password validation in the deployed application version enforced only the permitted password length (5–40 characters).

Table 5.2 Registration Validation Results

Test Case	Payload	Observation	Result
Invalid Email	abc@	Registration blocked with validation message	Passed
Weak Password	11111	Password length requirement enforced (5–40 characters)	Passed

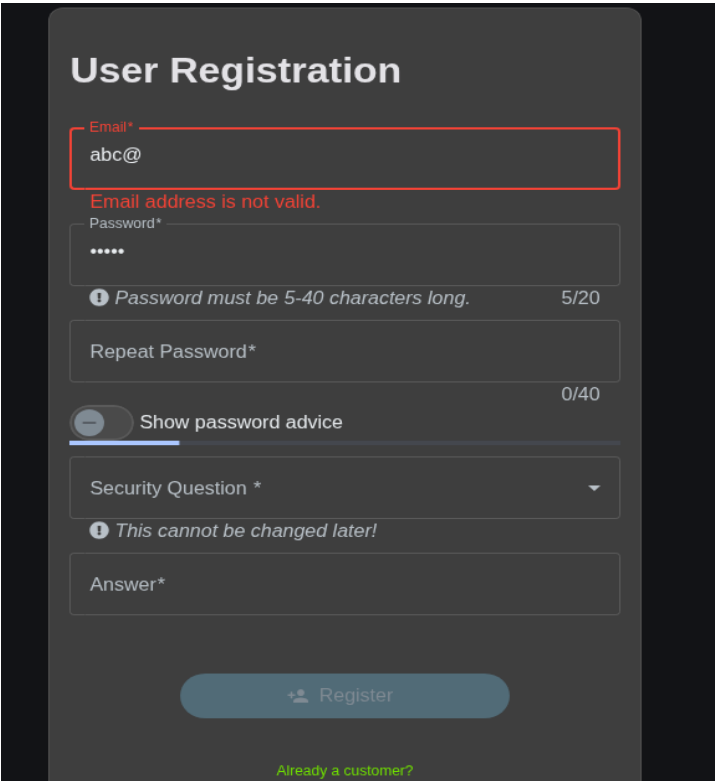


Figure 13 Invalid Email Validation

Figure Description

Figure 5.3 demonstrates client-side validation of an incorrectly formatted email address. The application prevented registration and displayed an appropriate validation message.

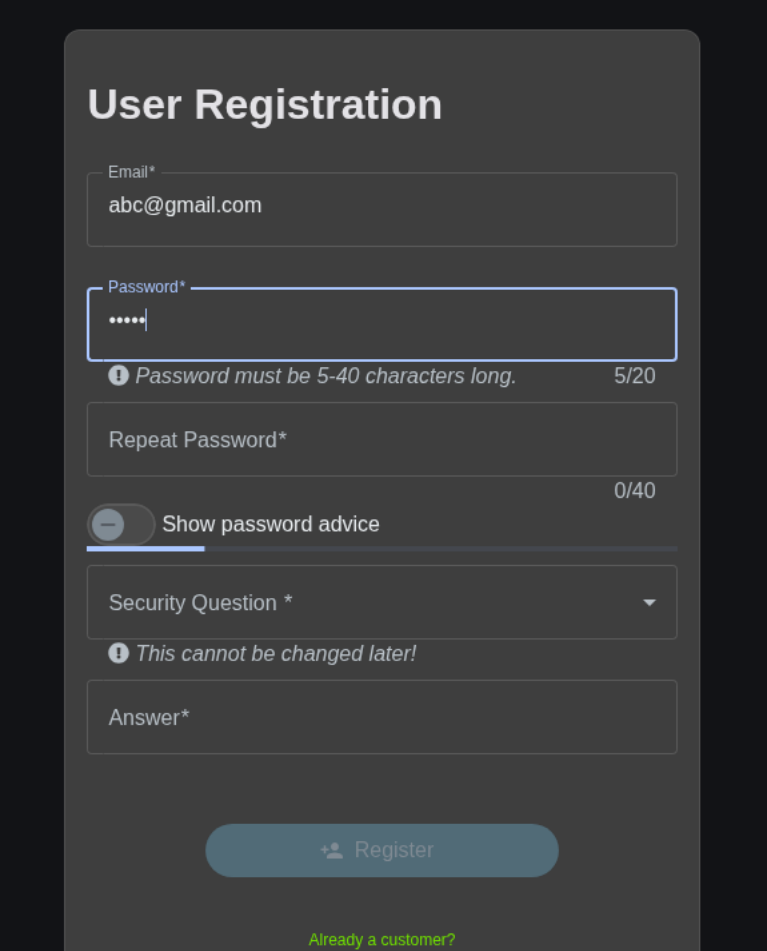
A screenshot of a 'User Registration' form. The form has a dark background with light-colored text and input fields. The title 'User Registration' is at the top. Below it, there are four input fields: 'Email*', 'Password*', 'Repeat Password*', and 'Answer*'. The 'Email*' field contains 'abc@gmail.com'. The 'Password*' field contains '....' and has a red border. Below the 'Password*' field, there is a red error message: 'Password must be 5-40 characters long.' with a character count '5/20'. Below the 'Repeat Password*' field, there is a character count '0/40'. There is a toggle switch labeled 'Show password advice'. Below that is a 'Security Question *' dropdown menu with a red error message: 'This cannot be changed later!'. At the bottom, there is a 'Register' button with a user icon and a link 'Already a customer?'.

Figure 14 Password Length Validation

Figure Description

Figure 5.4 shows the password validation mechanism. During testing, the application enforced the configured password length requirement of **5–40 characters**. No additional password complexity requirements were observed.

5.5. Search Input Validation

The product search functionality was evaluated using normal search terms and common security testing payloads, including SQL injection strings and special characters.

Burp Suite captured the search request submitted to the `/rest/products/search` endpoint. When the SQL injection test payload `' OR 1=1 --` was submitted, the application completed the request successfully and returned **"No results found"** without generating server errors or abnormal behavior.

Table 5.3 Search Validation Results

Payload	Observation	Result
Normal Search	Products returned successfully	Passed
' OR 1=1 --	No results returned; application remained stable	Passed

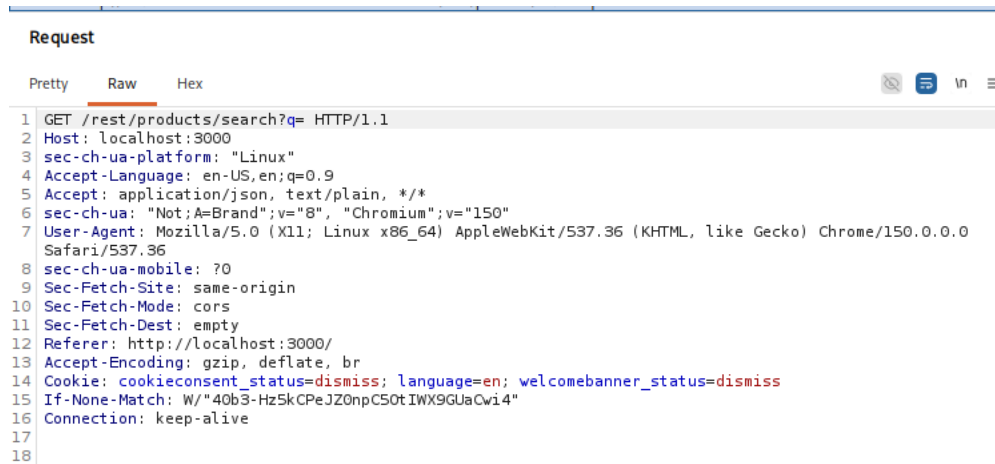


Figure 5.15 Product Search Request

Figure Description

Figure 5.5 shows the intercepted HTTP GET request submitted to the `/rest/products/search` endpoint. The request contains the user-supplied search parameter.

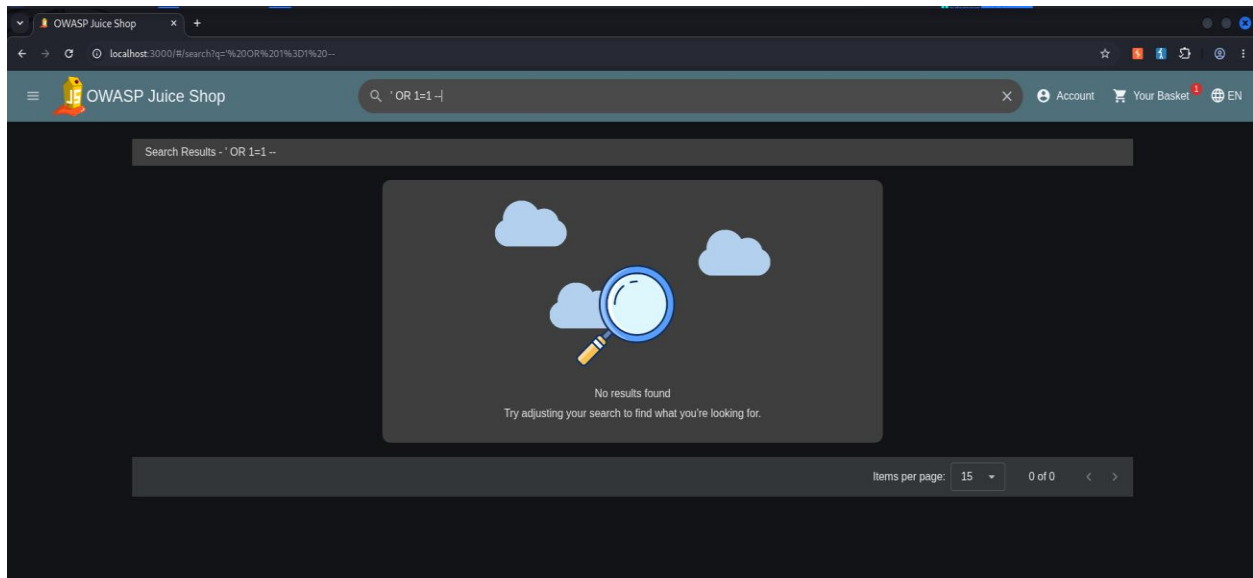


Figure 5.6 Product Search Response

Figure Description

Figure 5.6 presents the application's response after submitting the SQL injection test payload. The application returned **"No results found"** and continued operating normally without exposing server errors.

5.6. Address Input Validation

The address management functionality was evaluated by submitting special characters and a common SQL injection payload within the street address field.

Burp Suite confirmed that the payload was transmitted to the server as ordinary user input within the JSON request body.

Table 5.4 Address Validation Result

Payload	Observation	Result
' OR 1=1 --	Accepted as input; no abnormal server behavior observed	Passed

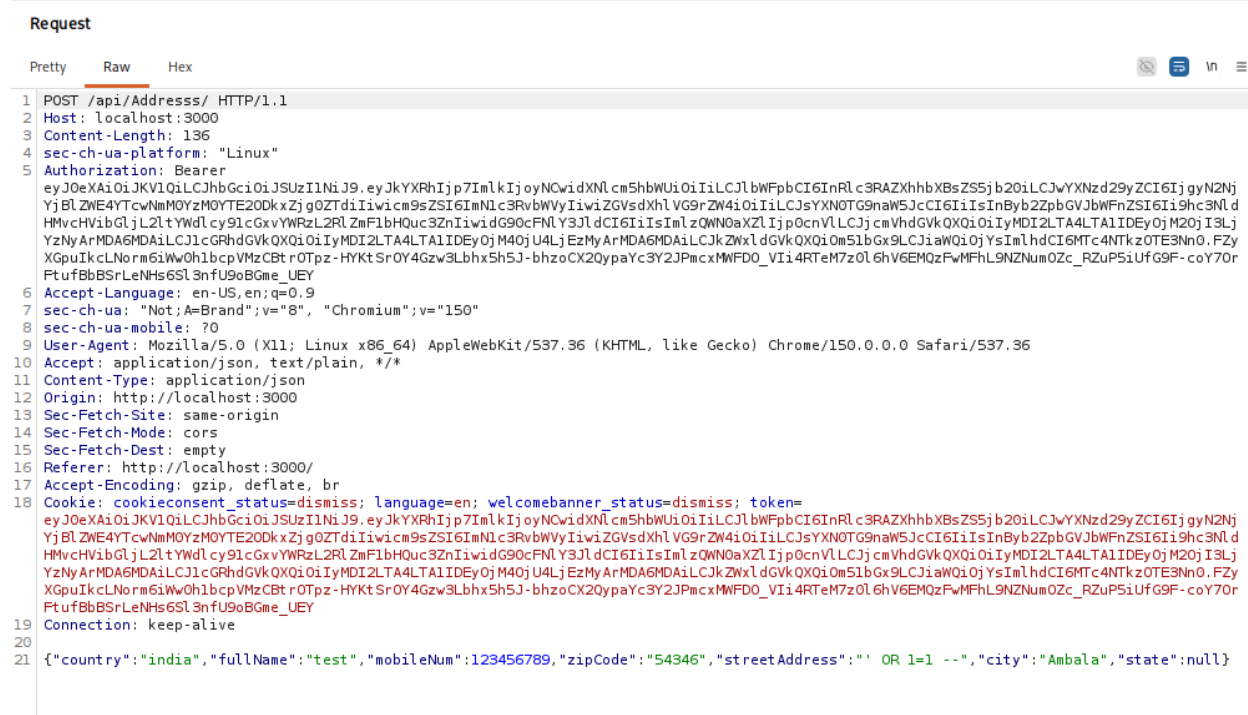


Figure 17 Address Submission Request

Figure Description

Figure 5.7 illustrates the intercepted address submission request. The injected test payload was transmitted as user input within the JSON body, and no server-side errors were observed during processing.

5.8. Input Validation Summary

Table 5.6 Overall Input Validation Assessment

Test Area	Observation	Assessment
Login	Generic authentication error displayed	Good Practice
Registration	Email format validated	Good Practice
Password	Length validation enforced	Acceptable
Search	SQLi test payload did not affect application	No abnormal behavior observed
Address	Special characters accepted as input without errors	Requires deeper server-side testing
Profile	Special characters URL-encoded before transmission	Normal browser behavior

6. OWASP Top 10 Security Assessment

6.1. Objective

The objective of this phase was to assess the security posture of the OWASP Juice Shop application with reference to the **OWASP Top 10 – 2021** Web Application Security Risks. The assessment combined manual testing using **Burp Suite** and **Nmap** with an automated passive security assessment performed using **OWASP ZAP**. The purpose was to identify observable security weaknesses, evaluate implemented security controls, and document the findings based on practical evidence collected during the audit.

6.2. Assessment Methodology

The assessment was conducted using both manual and automated testing techniques.

Manual Assessment

- Network reconnaissance using Nmap.
- Authentication and session management testing using Burp Suite.
- Input validation testing using Burp Suite.
- Verification of HTTP requests, responses, headers, cookies, and authentication tokens.

Automated Assessment

- Passive vulnerability scanning using OWASP ZAP v2.17.0.
- Automated crawling of application resources.
- Detection of configuration weaknesses and information disclosure issues.
- Identification of missing HTTP security headers.

Table 6.1 Tools Used During Security Assessment

Tool	Purpose
Kali Linux	Security testing environment
Nmap	Reconnaissance and service enumeration
Burp Suite Community Edition	Manual HTTP interception and analysis
OWASP ZAP v2.17.0	Automated vulnerability scanning

6.3. OWASP Top 10 Assessment Summary

Table 6.2 OWASP Top 10 Assessment Status

OWASP Category	Assessment Status	Remarks
A01 – Broken Access Control	Partially Assessed	Authentication and protected resource access verified

OWASP Category	Assessment Status	Remarks
A02 – Cryptographic Failures	Partially Assessed	Login request and token handling observed
A03 – Injection	Assessed	SQLi and XSS test payloads evaluated
A04 – Insecure Design	Not Assessed	Outside project scope
A05 – Security Misconfiguration	Assessed	Manual verification and ZAP findings
A06 – Vulnerable and Outdated Components	Not Assessed	Version analysis not performed
A07 – Identification & Authentication Failures	Assessed	Login, registration and session management tested
A08 – Software & Data Integrity Failures	Not Assessed	Outside project scope
A09 – Security Logging & Monitoring Failures	Not Assessed	Server logs unavailable
A10 – Server-Side Request Forgery (SSRF)	Not Assessed	No SSRF functionality identified

6.4. A01 – Broken Access Control

Objective

To verify whether protected resources could be accessed without valid authentication.

Assessment

Protected application resources were tested before and after user logout. Access to authenticated functionality required a valid session. After logout, accessing the user information endpoint returned an empty user object, indicating that the authenticated session was no longer active.

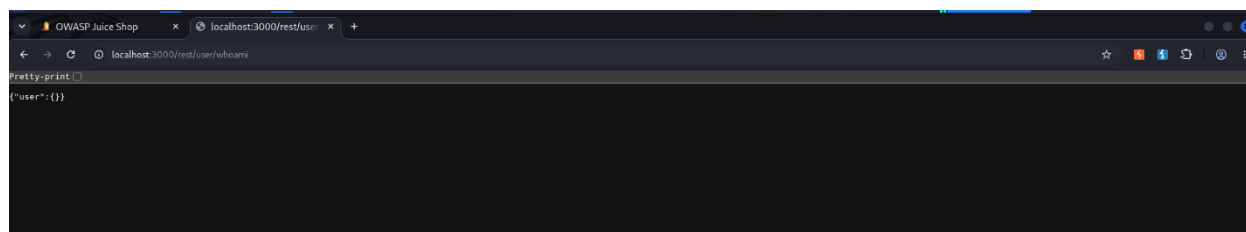


Figure 6.1 Access to Protected Resource After Logout

Figure Description

Figure 6.1 illustrates the application's response after logout when attempting to access the `/rest/user/whoami` endpoint. The application returned an empty user object, demonstrating that the previous authenticated session had been terminated successfully.

Observation

No evidence of unauthorized access to protected resources was observed during testing.

Result

Status: Partially Assessed

6.5. A02 – Cryptographic Failures

Objective

To examine how sensitive authentication information is transmitted between the client and the application.

Assessment

The intercepted login request showed that user credentials were transmitted within the HTTP POST request body in the controlled localhost laboratory environment. Authentication was subsequently maintained using a bearer token after successful login.



Figure 6.2 Login Request Captured in Burp Suite

Figure Description

Figure 6.2 shows the intercepted authentication request containing the user's login credentials submitted to the `/rest/user/login` endpoint during testing.

Observation

Bearer token–based authentication was implemented after successful login.

Result

Status: Partially Assessed

6.6. A03 – Injection

Objective

To evaluate the application's handling of potentially malicious user input.

Assessment

Multiple test payloads were submitted through user input fields, including:

- ' OR 1=1 --
- <script>alert(1)</script>
- Special characters (' , " , < , >)

The Product Search functionality accepted the test payloads as search input and returned "**No results found**" without producing application errors or executing the supplied scripts.

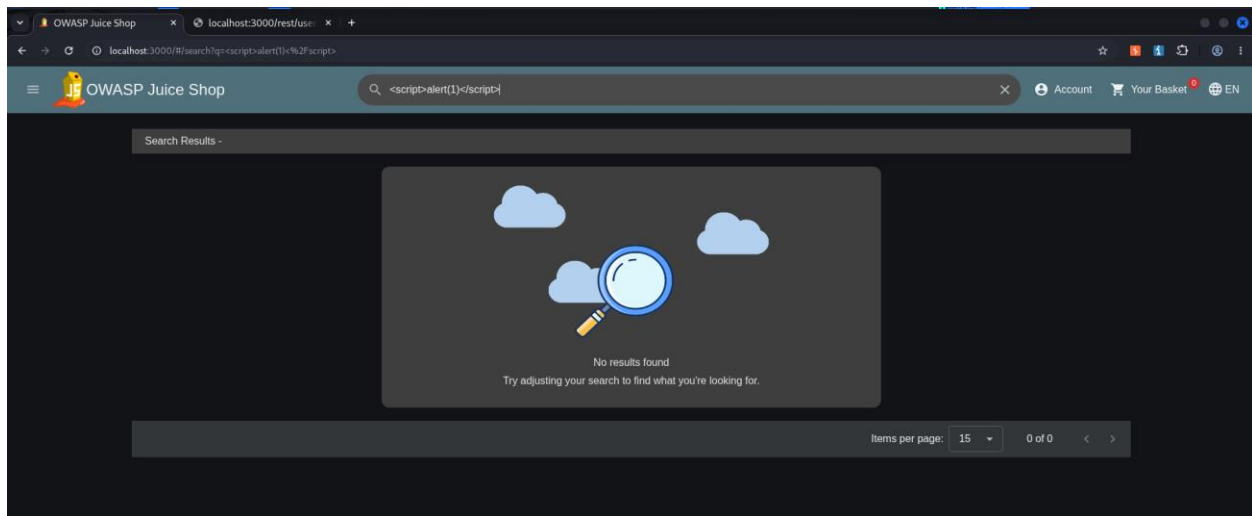


Figure 6.3 XSS Test Using Product Search

Figure Description

Figure 6.3 demonstrates the submission of a basic Cross-Site Scripting (XSS) payload through the product search feature. The application returned no matching results and did not execute the injected script.

Table 6.3 Injection Testing Summary

Payload	Observation	Result
' OR 1=1 --	No abnormal behaviour observed	Passed
<script>alert(1)</script>	Script not executed	Passed

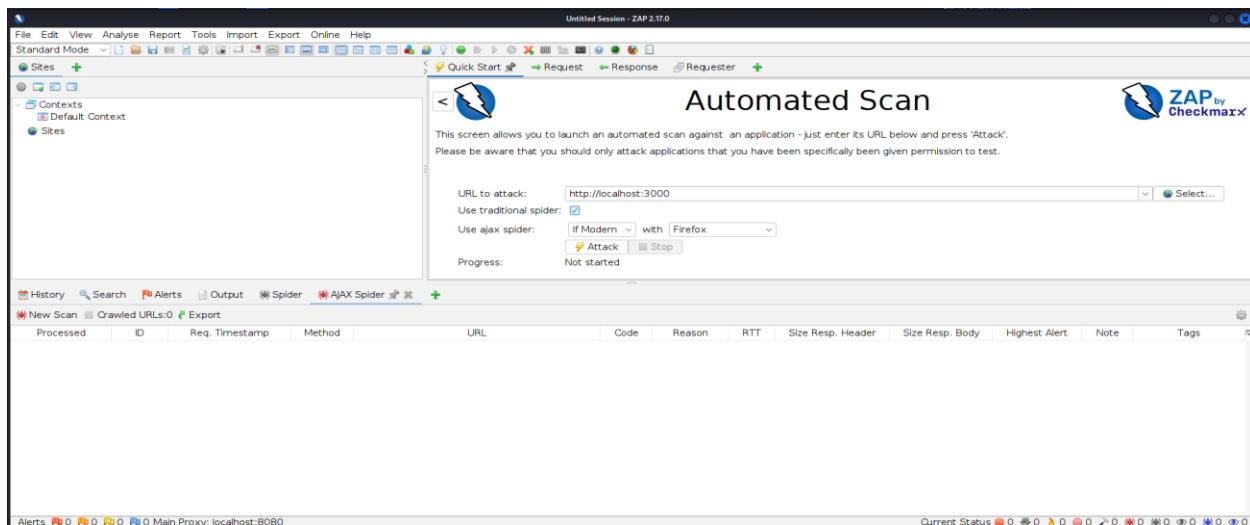


Figure 6.5 OWASP ZAP Quick Scan Configuration

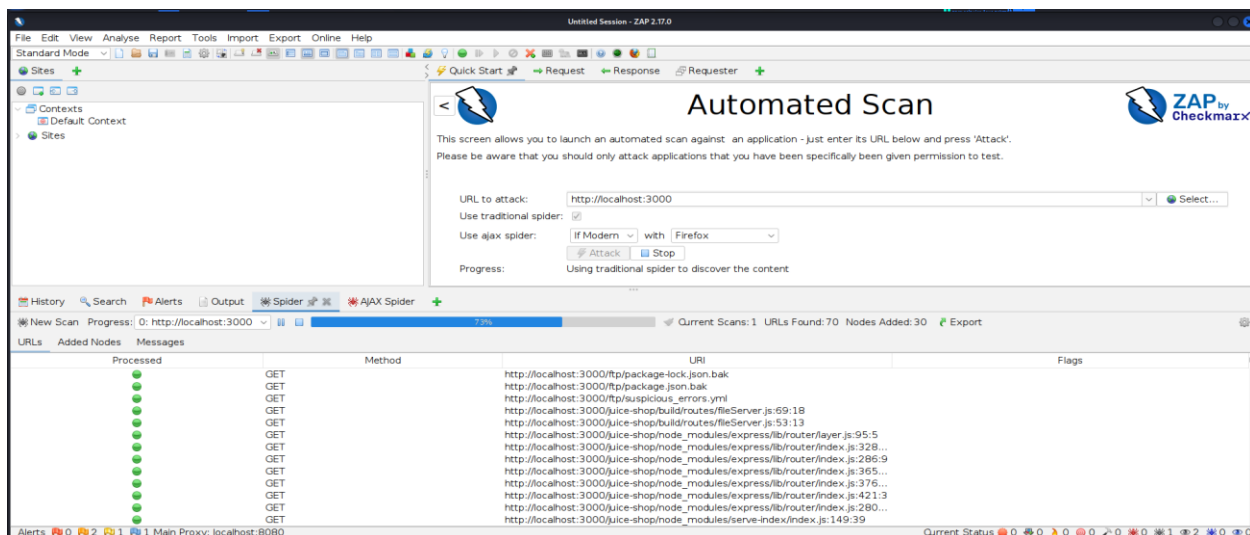


Figure 6.6 OWASP ZAP Scan in Progress

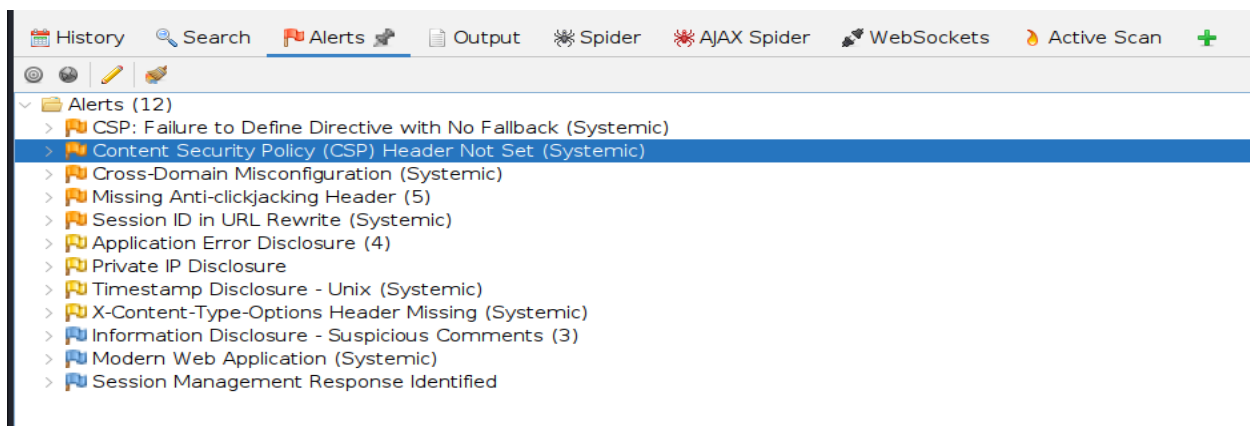


Figure 6.7 OWASP ZAP Alert Summary

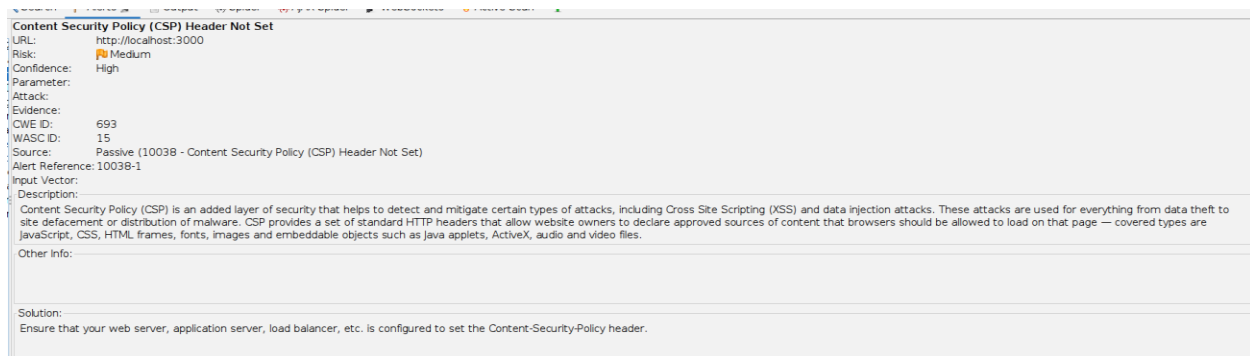


Figure 6.8 OWASP ZAP Alert Details

Table 6.4 Major Observations Reported by OWASP ZAP

Observation	Risk Level
Content Security Policy (CSP) Header Not Set	Medium
CSP Directive Without Fallback	Medium
Cross-Domain Misconfiguration	Medium
Missing Anti-clickjacking Header	Medium
Session ID in URL Rewrite	Medium
Application Error Disclosure	Low
Private IP Disclosure	Low
Timestamp Disclosure	Low
X-Content-Type-Options Header Missing	Low
Information Disclosure – Suspicious Comments	Informational
Session Management Response Identified	Informational

Observation

The automated scan primarily identified configuration-related security observations associated with HTTP security headers and application information disclosure. These findings supplement the manual assessment and provide additional insight into the application's security posture.

6.8. A07 – Identification and Authentication Failures

Objective

To evaluate the authentication and session management mechanisms.

Assessment

The authentication mechanism was tested through user registration, login, invalid authentication attempts, session verification, and logout.

The application:

- Successfully registered a new user.
- Authenticated valid users.
- Rejected invalid credentials.
- Displayed the generic message "**Invalid email or password.**"
- Used bearer token–based authentication.
- Properly terminated authenticated sessions after logout.

Table 6.5 Authentication Assessment Summary

Test	Result
User Registration	Successful
Valid Login	Successful
Invalid Login	Generic error displayed
Bearer Token Authentication	Verified
Logout	Successful
Session Termination	Verified

Observation

Authentication controls functioned as expected throughout the assessment and no evidence of authentication bypass was observed.

6.9. Overall Assessment Summary

Table 6.6 Overall OWASP Top 10 Assessment

Category	Assessment
Broken Access Control	Partially Assessed
Cryptographic Failures	Partially Assessed
Injection	Assessed
Security Misconfiguration	Assessed
Identification & Authentication Failures	Assessed

Category	Assessment
Remaining Categories	Not Assessed

6.10. Chapter Summary

This chapter evaluated the OWASP Juice Shop application against selected categories of the OWASP Top 10 (2021) through both manual and automated security testing. Manual assessment using Burp Suite and Nmap verified authentication behavior, session management, and input validation, while OWASP ZAP identified several configuration-related observations, including missing Content Security Policy headers, clickjacking protection issues, and information disclosure findings. Overall, the assessment demonstrated that core authentication mechanisms functioned correctly, input validation handled the tested payloads without abnormal behavior, and automated scanning highlighted areas where security configuration could be improved. These observations form the basis of the detailed findings and remediation recommendations presented in the subsequent chapters.

7. Findings and Risk Analysis

This chapter is what your mentor will focus on because it summarizes **what was found**, **how severe it is**, and **how it should be fixed**.

Since this is a **security audit**, we should **not** claim that every ZAP alert is a confirmed vulnerability. Some ZAP alerts are informational or require manual verification. Therefore, we'll classify them based on your manual testing and ZAP results.

7.1. Overview

The findings presented in this chapter are based on the manual security assessment performed using **Burp Suite Community Edition** and **Nmap**, together with the automated passive vulnerability assessment conducted using **OWASP ZAP**. Each finding has been evaluated according to its potential security impact, supporting evidence, and recommended remediation measures. Risk ratings are assigned based on the severity of the observation and its potential effect on the application's security posture.

7.2. Finding F-01

Missing Content Security Policy (CSP) Header

Finding ID

F-01

OWASP Category

A05 – Security Misconfiguration

Risk Rating

Medium

Description

OWASP ZAP identified that the application responses do not include a **Content-Security-Policy (CSP)** header. A CSP header restricts the sources from which browsers are allowed to load scripts, stylesheets, images, and other resources, thereby reducing the impact of client-side attacks such as Cross-Site Scripting (XSS).

Evidence

The automated OWASP ZAP scan reported "Content Security Policy (CSP) Header Not Set" with Medium Risk and High Confidence.

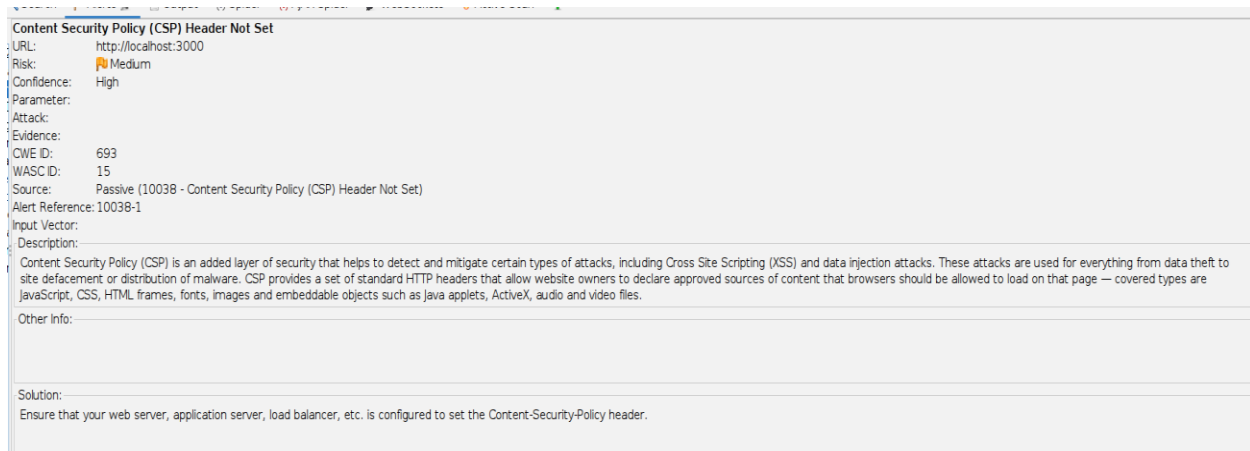


Figure 7.1 Missing Content Security Policy Header

Figure Description

Figure 7.1 displays the OWASP ZAP alert identifying the absence of the Content-Security-Policy (CSP) header in the HTTP response. The alert provides the associated risk level, confidence rating, description, and recommended mitigation.

Impact

Without a CSP header, browsers cannot enforce restrictions on trusted content sources. If an XSS vulnerability exists elsewhere in the application, its impact may be significantly increased.

Recommendation

Configure a Content-Security-Policy header defining trusted sources for scripts, styles, fonts, images, and other web resources.

7.3. Finding F-02

Missing Anti-Clickjacking Protection

Finding ID

F-02

OWASP Category

A05 – Security Misconfiguration

Risk Rating

Medium

Description

The automated scan identified missing anti-clickjacking protection for application pages. Without proper frame protection, attackers may attempt to embed application pages within malicious websites.

Evidence

The finding was identified during the automated OWASP ZAP assessment.

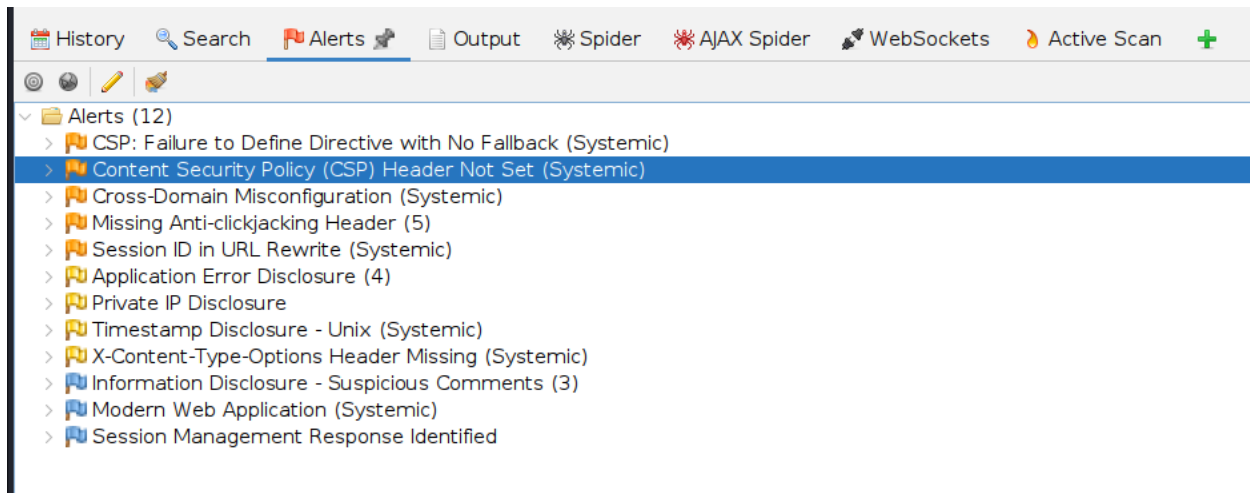


Figure 7.2 OWASP ZAP Alert Summary

Figure Description

Figure 7.2 presents the summary of alerts generated by OWASP ZAP. The alert list includes missing anti-clickjacking protection among the identified security observations.

Impact

Attackers may exploit missing frame protection to perform clickjacking attacks that trick users into executing unintended actions.

Recommendation

Configure either:

- X-Frame-Options
- or
- Content-Security-Policy: frame-ancestors

to prevent unauthorized framing of application pages.

7.4. Finding F-03

Authentication Mechanism Verification

Finding ID

F-03

OWASP Category

A07 – Identification and Authentication Failures

Risk Rating

Informational

Description

Manual authentication testing confirmed that the application correctly authenticated valid users, rejected invalid credentials, and maintained authenticated sessions using token-based authentication.

Evidence

Burp Suite successfully intercepted the authentication request submitted to the /rest/user/login endpoint.

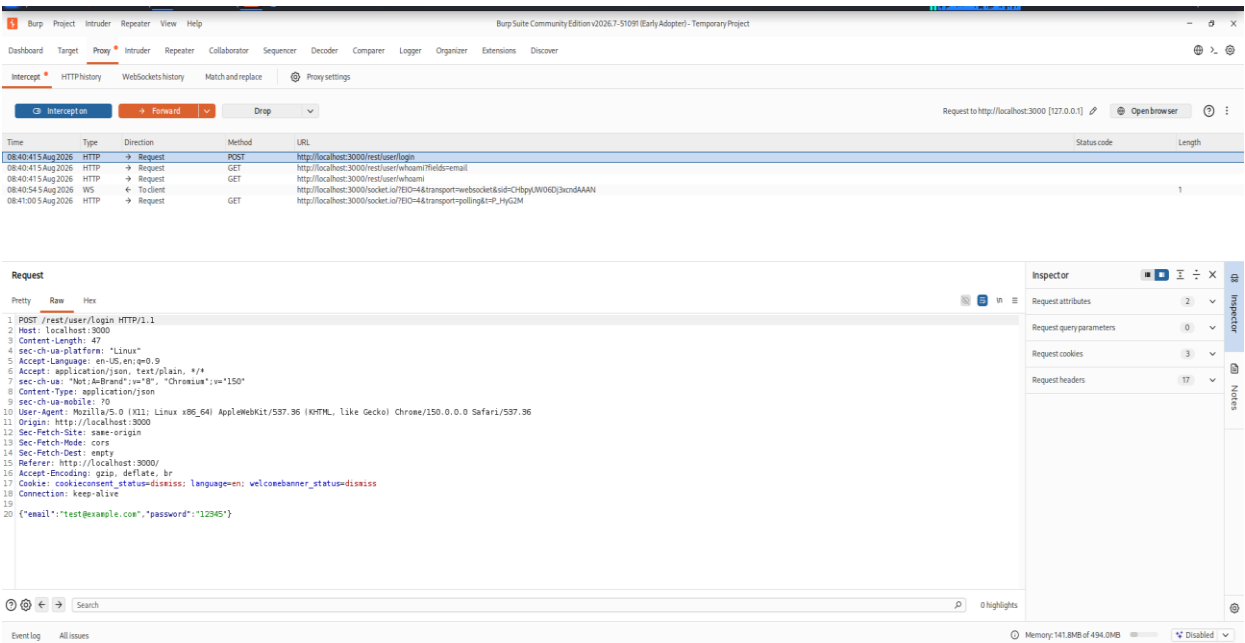


Figure 7.3 Authentication Request Captured in Burp Suite

Figure Description

Figure 7.3 illustrates the intercepted login request captured by Burp Suite during authentication testing. The request demonstrates the transmission of user credentials to the authentication endpoint.

Additional Evidence

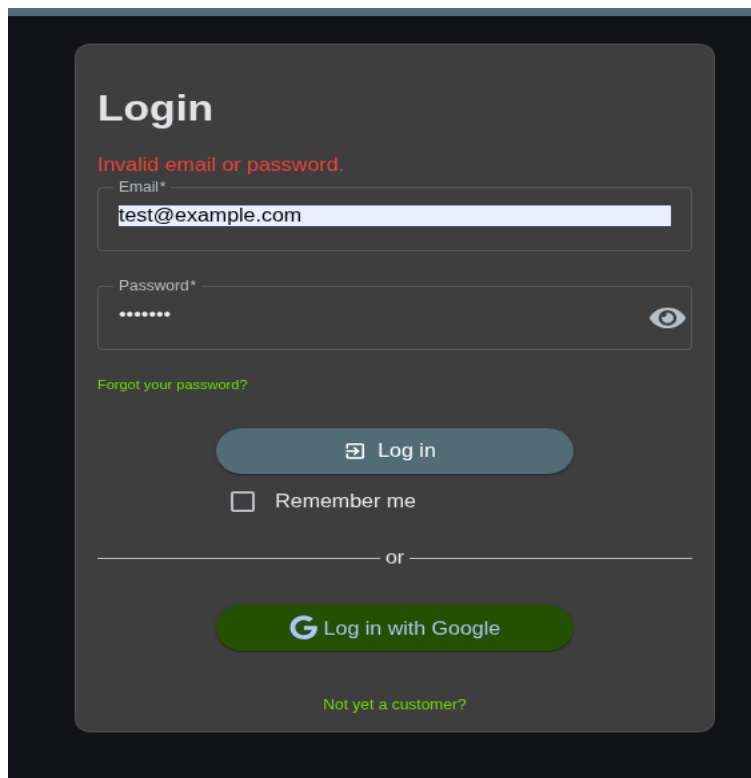


Figure 7.4 Authentication Failure Message

Figure Description

Figure 7.4 shows the generic authentication error displayed after submitting invalid credentials. The application does not reveal whether the username or password is incorrect, reducing the possibility of user enumeration.

Impact

No authentication bypass or information leakage was observed during testing.

Recommendation

Continue using generic authentication failure messages and periodically review authentication controls and password policies.

7.5. Finding F-04

Input Validation Assessment

Finding ID

F-04

OWASP Category

A03 – Injection

Risk Rating

Informational

Description

Multiple SQL Injection and Cross-Site Scripting (XSS) test payloads were submitted through user-accessible input fields during manual testing.

The application processed the submitted payloads without executing injected scripts or generating abnormal server responses.

Evidence

Manual testing included payloads such as:

- ' OR 1=1 --
- <script>alert(1)</script>

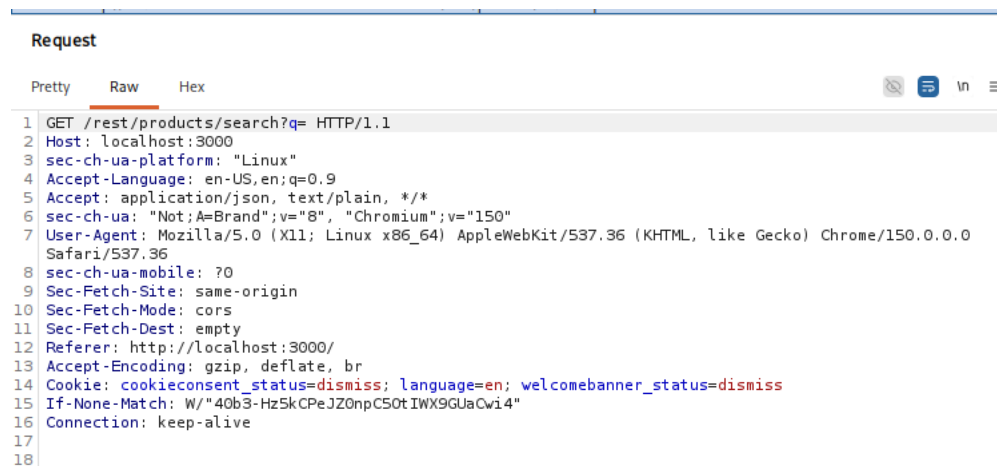


Figure 7.5 SQL Injection Test Payload Submitted Through Product Search

Figure Description

Figure 7.5 illustrates the intercepted search request containing a SQL Injection test payload submitted to the /rest/products/search endpoint.

Additional Evidence

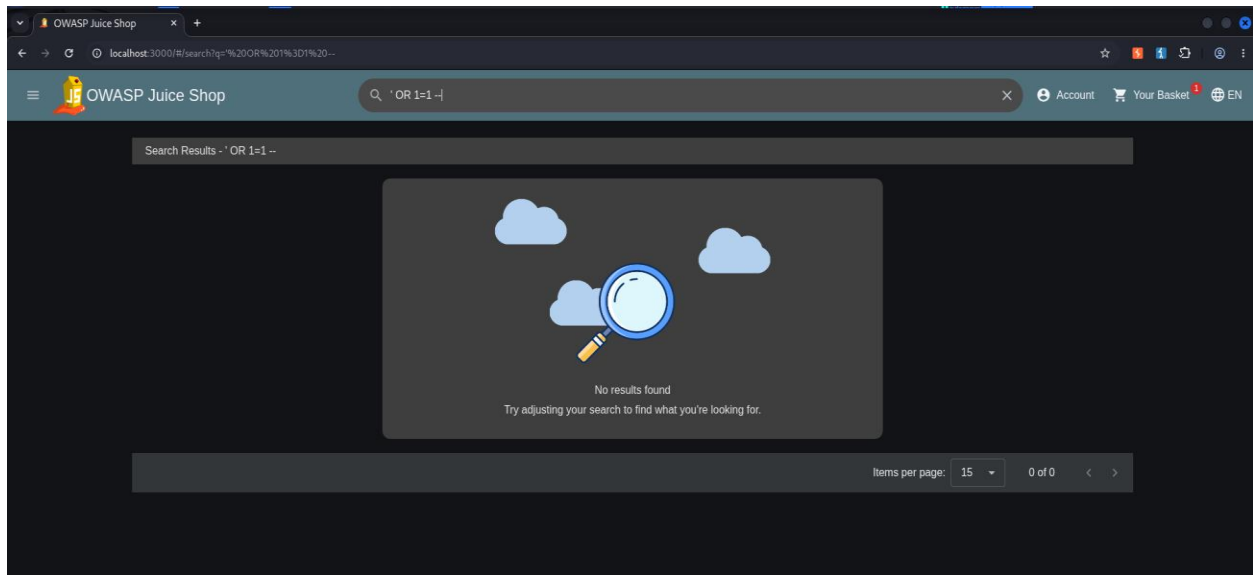


Figure 7.6 Search Response

Figure Description

Figure 7.6 shows the application's response after the SQL Injection test payload was submitted. The application returned **"No results found"** and continued operating normally without exposing server-side errors.

Impact

No successful SQL Injection or reflected Cross-Site Scripting vulnerability was observed during the assessment.

Recommendation

Continue implementing:

- Server-side validation
- Parameterized database queries
- Output encoding
- Secure input handling practices

7.6. Finding F-05

Session Management Verification

Finding ID

F-05

OWASP Category

A01 – Broken Access Control

Risk Rating

Low

Description

Session management was evaluated by verifying authenticated access before and after user logout. Following logout, access to the protected `/rest/user/whoami` endpoint returned an empty user object, indicating that the authenticated session had been successfully terminated.

Evidence

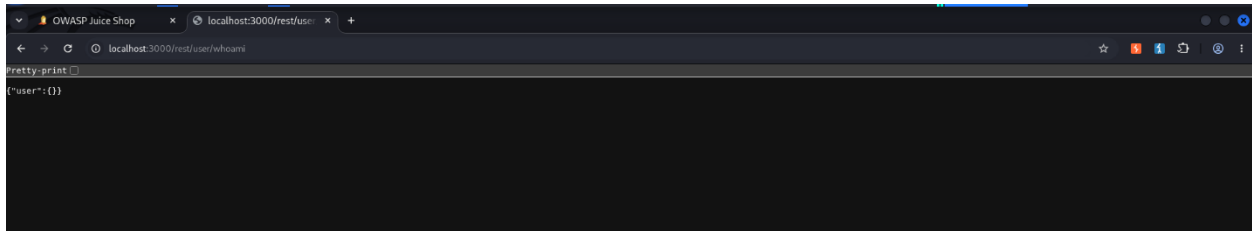


Figure 7.7 Access to Protected Resource After Logout

Figure Description

Figure 7.7 demonstrates the application's response after logout when a protected endpoint was accessed. The response confirmed that the previous authenticated session was no longer valid.

Impact

No evidence of unauthorized access to protected resources was observed during testing.

Recommendation

Continue implementing secure session termination and periodically review session timeout and token expiration policies.

7.7. Risk Summary

Table 7.1 Findings Summary

Finding ID	Finding	OWASP Category	Risk
F-01	Missing Content Security Policy Header	A05	Medium
F-02	Missing Anti-Clickjacking Protection	A05	Medium
F-03	Authentication Mechanism Verification	A07	Informational
F-04	Input Validation Assessment	A03	Informational
F-05	Session Management Verification	A01	Low

7.8. Risk Distribution

Table 7.2 Risk Level Distribution

Risk Level	Number of Findings
High	0
Medium	2
Low	1
Informational	2

7.9. Chapter Summary

The findings identified during the security assessment were derived from both manual testing and automated vulnerability scanning. Manual verification confirmed that the application implemented appropriate authentication behavior, session management, and input handling for the tested scenarios. The OWASP ZAP scan highlighted several configuration-related observations, primarily involving missing HTTP security headers and browser security controls. Although no critical vulnerabilities were identified within the scope of this assessment, the recommended remediation measures will further strengthen the application's security posture and reduce its exposure to common web application attacks.

8. Remediation Recommendations

8.1. Overview

The purpose of remediation is to reduce the security risks identified during the assessment and strengthen the overall security posture of the application. Based on the findings presented in Chapter 7, several corrective measures are recommended to address configuration weaknesses, improve security controls, and align the application with industry best practices and the OWASP Top 10 (2021) guidelines.

The recommendations presented in this chapter prioritize practical improvements that can be implemented during application development and deployment. The recommendations are categorized according to their associated findings and implementation priority.

8.2. Remediation Matrix

Table 8.1 Remediation Recommendations

Finding ID	Security Observation	Recommended Remediation	Priority
F-01	Missing Content Security Policy (CSP) Header	Configure an appropriate Content-Security-Policy (CSP) header to define trusted sources for scripts, styles, images, fonts, and other resources.	High
F-02	Missing Anti-Clickjacking Protection	Configure X-Frame-Options: DENY or implement the frame-ancestors directive within the CSP header.	High
F-03	Authentication Mechanism	Continue using generic authentication error messages, strengthen password policies, implement account lockout mechanisms, and periodically review session management.	Medium
F-04	Input Validation	Validate all user input on the server side, sanitize user-supplied data, use parameterized queries, and apply output encoding before rendering user input.	High
F-05	Session Management	Configure secure session expiration, invalidate authentication tokens after logout, and enable Secure, HttpOnly, and SameSite cookie attributes where applicable.	Medium

8.3. Detailed Remediation Recommendations

8.3.1. Configure Content Security Policy (CSP)

The automated security assessment identified the absence of a **Content Security Policy (CSP)** header. Implementing a properly configured CSP reduces the risk of Cross-Site Scripting (XSS) attacks by restricting the sources from which browsers are permitted to load content.

Recommended Actions

- Configure the Content-Security-Policy HTTP response header.
- Restrict JavaScript execution to trusted domains.
- Limit inline script execution where possible.
- Define trusted sources for stylesheets, fonts, images, and media resources.

Expected Benefit

- Reduced exposure to client-side code injection attacks.
- Improved browser-enforced security controls.

8.3.2. Implement Clickjacking Protection

The assessment identified missing clickjacking protection for certain application responses.

Recommended Actions

- Configure the X-Frame-Options response header.
- Alternatively, configure the frame-ancestors directive within the Content Security Policy.
- Prevent application pages from being embedded within unauthorized websites.

Expected Benefit

- Protection against clickjacking attacks.
- Reduced risk of unauthorized user interaction.

8.3.3. Strengthen Authentication Controls

Manual testing confirmed that authentication mechanisms functioned correctly. However, additional security controls are recommended to further strengthen account protection.

Recommended Actions

- Enforce stronger password complexity requirements.
- Implement account lockout after repeated failed login attempts.
- Require multi-factor authentication (MFA) for privileged accounts where applicable.
- Periodically review session timeout values.
- Continue using generic authentication failure messages.

Expected Benefit

- Improved protection against brute-force attacks.
- Reduced user enumeration risk.

- Enhanced authentication security.

8.3.4. Improve Input Validation

Although the tested SQL Injection and Cross-Site Scripting payloads did not produce abnormal application behavior during the assessment, secure input handling should continue to be enforced throughout the application.

Recommended Actions

- Validate all user input on the server side.
- Use allow-list input validation wherever possible.
- Apply output encoding before displaying user-generated content.
- Use parameterized SQL queries and prepared statements.
- Sanitize user input before processing.

Expected Benefit

- Reduced risk of SQL Injection.
- Improved protection against Cross-Site Scripting (XSS).
- More robust handling of unexpected input.

8.3.5. Enhance Session Management

The application successfully terminated user sessions after logout. Additional security improvements can further strengthen session protection.

Recommended Actions

- Invalidate authentication tokens immediately after logout.
- Configure reasonable session expiration times.
- Enable Secure, HttpOnly, and SameSite cookie attributes where applicable.
- Rotate authentication tokens after successful login.
- Monitor session activity for abnormal behavior.

Expected Benefit

- Reduced risk of session hijacking.
- Improved authentication lifecycle management.
- Stronger protection for authenticated users.

8.4. General Security Best Practices

In addition to the specific remediation measures described above, the following best practices are recommended:

- Perform regular vulnerability assessments using OWASP ZAP and Burp Suite.
- Conduct periodic penetration testing after significant application updates.
- Keep third-party libraries and application dependencies up to date.
- Implement centralized security logging and monitoring.

- Perform secure code reviews during development.
- Integrate security testing within the software development lifecycle (SDLC).
- Follow the OWASP Secure Coding Practices and OWASP Top 10 recommendations.
- Regularly review application configuration for unnecessary information disclosure.
- Maintain an effective patch management process.
- Provide secure coding awareness training for developers.

8.5. Remediation Priority

Table 8.2 Implementation Priority

Priority Level	Recommended Activities
High	Configure Content Security Policy (CSP), implement clickjacking protection, strengthen input validation, and sanitize user input.
Medium	Improve authentication controls, enhance session management, and minimize information disclosure.
Low	Perform periodic security reviews, dependency updates, and continuous vulnerability assessments.

8.6. Expected Security Improvements

Implementing the recommended remediation measures is expected to provide the following security benefits:

- Improved protection against Cross-Site Scripting (XSS).
- Reduced exposure to clickjacking attacks.
- Stronger browser security through HTTP security headers.
- Enhanced authentication and session management.
- More secure handling of user-supplied input.
- Reduced information disclosure.
- Improved compliance with OWASP Top 10 security recommendations.
- Strengthened overall security posture of the web application.

8.7. Chapter Summary

This chapter presented practical remediation recommendations based on the findings identified during the security assessment. The recommendations focus on improving HTTP security configuration, strengthening authentication and session management, enhancing input validation, and reducing information disclosure. Implementing these measures will help improve the application's resilience against common web application attacks and align its security controls with recognized industry best practices. Regular security assessments, continuous monitoring, and secure development practices should be maintained to ensure long-term application security.

9. Conclusion

9.1. Conclusion

This project presented a structured security assessment of the **OWASP Juice Shop** web application within a controlled laboratory environment. The assessment successfully achieved its primary objective of evaluating the application's security posture through reconnaissance, application mapping, authentication and session testing, input validation assessment, and evaluation against selected categories of the **OWASP Top 10 (2021)**.

The assessment combined **manual security testing** using **Burp Suite Community Edition** and **Nmap** with **automated vulnerability scanning** using **OWASP ZAP**. This approach provided a comprehensive understanding of the application's behavior and enabled the identification of configuration-related security observations, authentication characteristics, session management practices, and input validation mechanisms.

Network reconnaissance confirmed that the target application was successfully deployed and accessible within the laboratory environment. Manual testing demonstrated that the application correctly handled user registration, authentication, session termination, and invalid login attempts. Input validation testing showed that the application appropriately handled the tested SQL injection and Cross-Site Scripting payloads without exhibiting abnormal behavior during the assessment. Automated scanning using OWASP ZAP identified several configuration-related observations, including missing HTTP security headers and information disclosure issues, which were documented and analyzed throughout the report.

Overall, the assessment indicates that while the application demonstrates appropriate behavior in several tested areas, opportunities remain to improve its security configuration through the implementation of additional HTTP security headers, reduction of information disclosure, and adoption of recommended secure configuration practices. These improvements would further strengthen the application's resilience against common web application attacks.

This project also provided practical experience in applying industry-standard web application security assessment methodologies using tools such as **Kali Linux**, **Nmap**, **Burp Suite Community Edition**, and **OWASP ZAP**. The assessment reinforced the importance of combining manual verification with automated scanning to produce accurate, evidence-based security findings and practical remediation recommendations.

9.2. Key Achievements

The following objectives were successfully completed during the assessment:

- Successfully deployed OWASP Juice Shop in a controlled laboratory environment.
- Configured Burp Suite Community Edition for HTTP interception and traffic analysis.
- Performed network reconnaissance using Nmap.
- Conducted application mapping and identified major application endpoints.
- Evaluated authentication and session management mechanisms.

- Performed input validation testing across multiple application components.
- Conducted an automated security assessment using OWASP ZAP.
- Identified and documented security observations with supporting evidence.
- Prepared a vulnerability matrix and remediation recommendations.
- Produced a structured security audit report based on practical findings.

9.3. Limitations

The assessment was conducted within a predefined project scope and therefore has the following limitations:

- Testing was limited to the locally deployed OWASP Juice Shop application.
- Denial-of-Service (DoS) testing was not performed.
- Source code review was outside the scope of this assessment.
- Infrastructure and operating system security were not evaluated.
- Server-side logging and monitoring mechanisms could not be verified.
- Some OWASP Top 10 categories require deeper penetration testing and source-level analysis, which were beyond the scope of this project.

9.4. Future Scope

Future security assessments may include the following activities:

- Comprehensive penetration testing covering all OWASP Top 10 categories.
- Secure code review and static application security testing (SAST).
- Dynamic application security testing (DAST) with authenticated scanning.
- API security assessment.
- Business logic testing.
- Dependency and third-party library analysis.
- Continuous vulnerability monitoring integrated into the development lifecycle.
- DevSecOps integration for automated security testing during software development.

9.5. Final Remarks

The Security Audit of OWASP Juice Shop successfully demonstrated the application of standard web application security assessment techniques within a controlled environment. By combining reconnaissance, manual testing, automated scanning, and structured documentation, the project provided practical experience in identifying, analyzing, and documenting security observations while emphasizing ethical testing practices and evidence-based reporting. The knowledge and skills gained through this assessment contribute to a stronger understanding of modern web application security and industry-standard security assessment methodologies.

REFERENCES

The following references were consulted during the planning, execution, and documentation of the security assessment.

1. OWASP Foundation. (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Retrieved from <https://owasp.org/www-project-top-ten/>
2. OWASP Foundation. *OWASP Juice Shop Documentation*. Retrieved from <https://owasp.org/www-project-juice-shop/>
3. PortSwigger Ltd. *Burp Suite Documentation*. Retrieved from <https://portswigger.net/burp/documentation>
4. The OWASP Foundation. *OWASP ZAP Documentation*. Retrieved from <https://www.zaproxy.org/docs/>
5. Gordon Lyon. *Nmap Reference Guide*. Retrieved from <https://nmap.org/book/>
6. Docker Inc. *Docker Documentation*. Retrieved from <https://docs.docker.com/>
7. Mozilla Developer Network (MDN). *HTTP Security Headers*. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
8. MITRE Corporation. *Common Weakness Enumeration (CWE)*. Retrieved from <https://cwe.mitre.org/>
9. Open Web Application Security Project (OWASP). *Web Security Testing Guide (WSTG)*. Retrieved from <https://owasp.org/www-project-web-security-testing-guide/>
10. National Institute of Standards and Technology (NIST). *Secure Software Development Framework (SSDF)*. Retrieved from <https://csrc.nist.gov/>

APPENDIX

Appendix A

Commands Used During Assessment

Table A.1 Commands Used

Tool	Command	Purpose
Docker	<code>docker run -d -p 3000:3000 --name juice-shop bkimminich/juice-shop</code>	Deploy OWASP Juice Shop
Docker	<code>docker ps</code>	Verify running containers
Docker	<code>docker start juice-shop</code>	Start existing container
Docker	<code>docker stop juice-shop</code>	Stop container
Docker	<code>docker rm juice-shop</code>	Remove container
Nmap	<code>nmap -sV localhost</code>	Service version detection
Nmap	<code>nmap -A localhost</code>	Aggressive scan
Burp Suite	Proxy Intercept	Capture HTTP requests
OWASP ZAP	Quick Scan	Automated vulnerability assessment

Appendix B

Tool Versions

Table B.1 Software Versions

Tool	Version
Kali Linux	2026.3
Docker	28.x
Burp Suite Community Edition	Community Edition
OWASP ZAP	2.17.0
Nmap	7.99
Mozilla Firefox	Latest

Appendix C

Project Folder Structure

Project2_Security_Audit_OWASP_Juice_Shop/

```
|
|
|— Audit_Report/
|   └─ Security_Audit_Report.pdf
|
|
|— Presentation_and_Demonstration/
|   └─ Security_Audit_Presentation.pptx
|
|
|— Vulnerability_Matrix/
|   └─ Vulnerability_Matrix.pdf
|
|
|— Screenshots/
|   └─ Lab_Setup/
|   └─ Reconnaissance/
|   └─ Authentication_and_Session_Testing/
|   └─ Input_Validation/
|   └─ OWASP_Top10/
|   └─ OWASP_ZAP/
|
|
└─ README.md
```

Appendix D

Ethical Statement

This security assessment was conducted exclusively within an authorized laboratory environment using **OWASP Juice Shop**, an intentionally vulnerable web application developed for security education and training. No unauthorized systems, production environments, or third-party services were targeted during this project. All testing activities were performed in accordance with ethical security testing practices and solely for educational purposes.